

Advanced Computer Graphics

Hierarchical Animaion

Homework

- Programming Exercise
 - Animating a rigid object (eg: BOX)
- Implement simple interpolation based animation
 - It could be presented as a limited number of control points
- Using at least two different interpolation methods
- Be able to add/modify control points for interpolation

Review

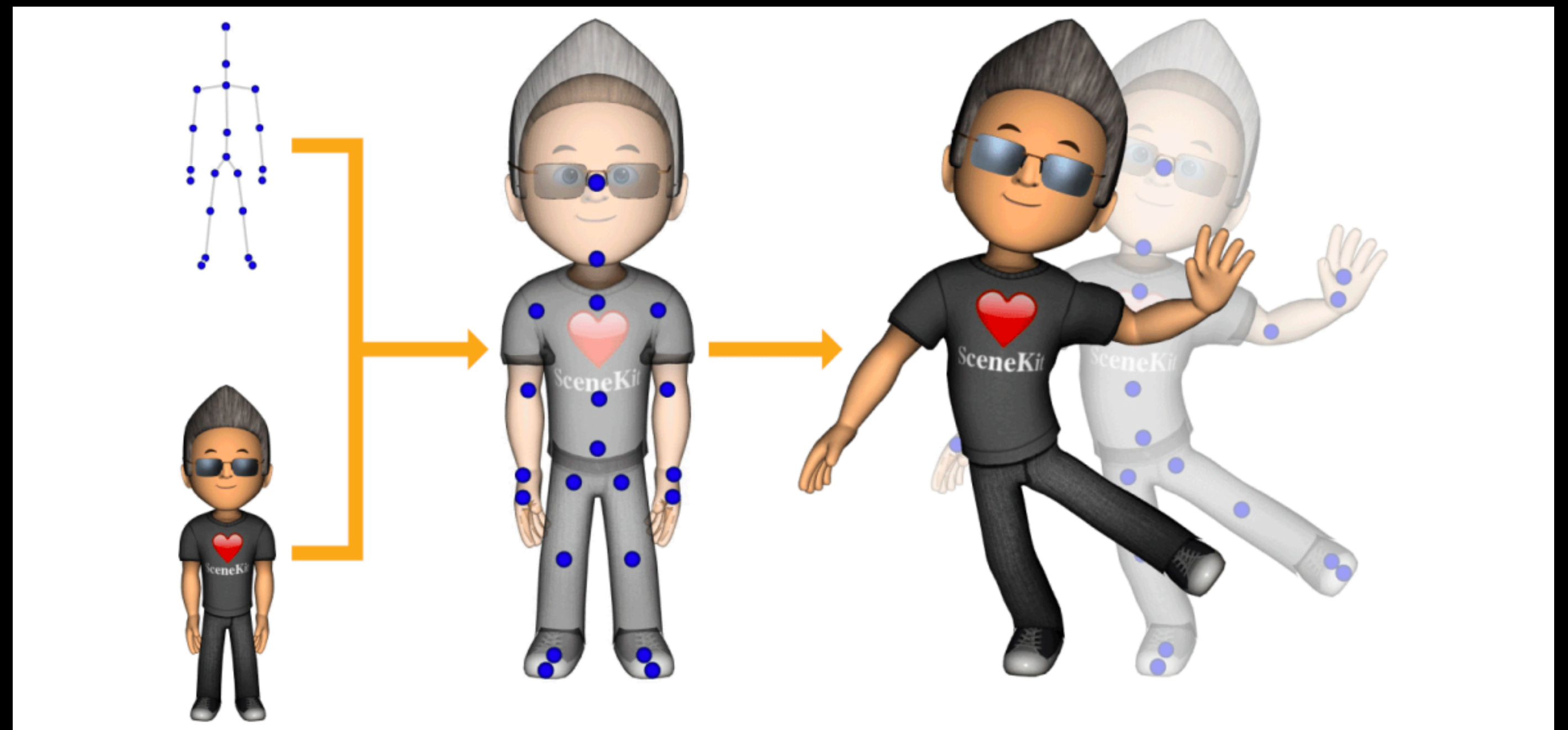
- Interpolation methods
- Interpolation based animation: Keyframing
 - Rigid body animation
 - Deformable animation

Introduction

- Now we can animate an object
 - How about a complex object
 - such as an animal or a human

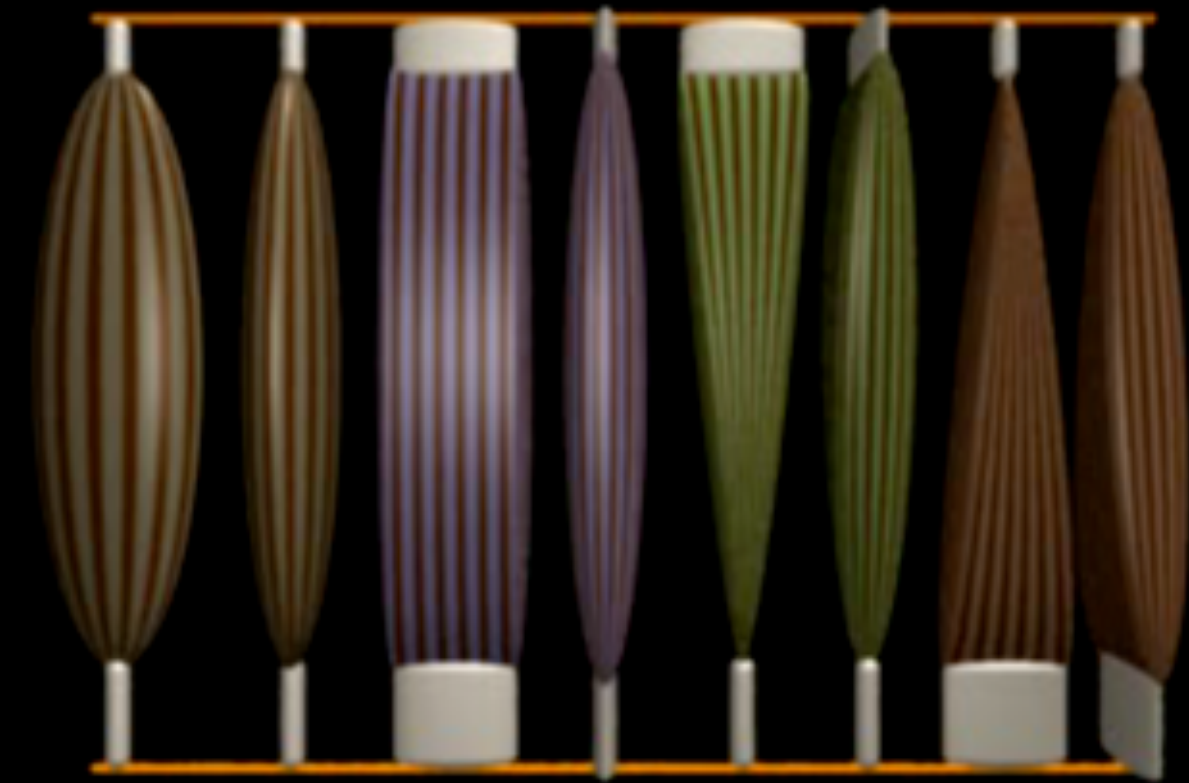
Introduction

- Characteristics of complex object
 - Component Movement
 - Hierarchical Movement
- Not easy to interpolate
- Resolution?



Introduction

- Character example
 - Key elements in most animation
- How to animate?
 - Physical laws?



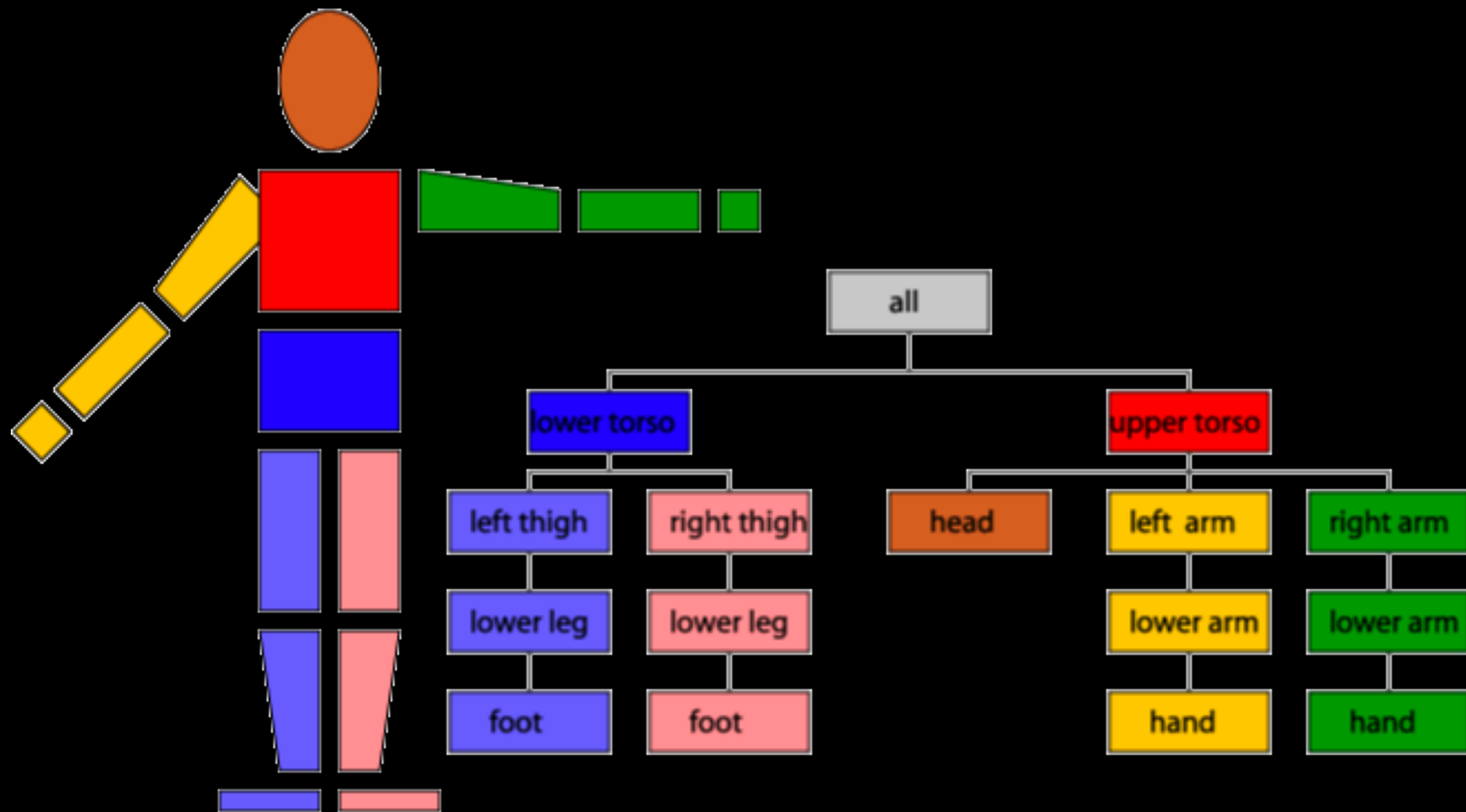
*Animations from Ferdi Scheepers, Richard E. Parent, Wayne E. Carlson, Stephen F. May
,Anatomy-Based Modeling of the Human Musculature, in Proceedings of SIGGRAPH '97*

HyungSeok Kim, Konkuk University

Introduction

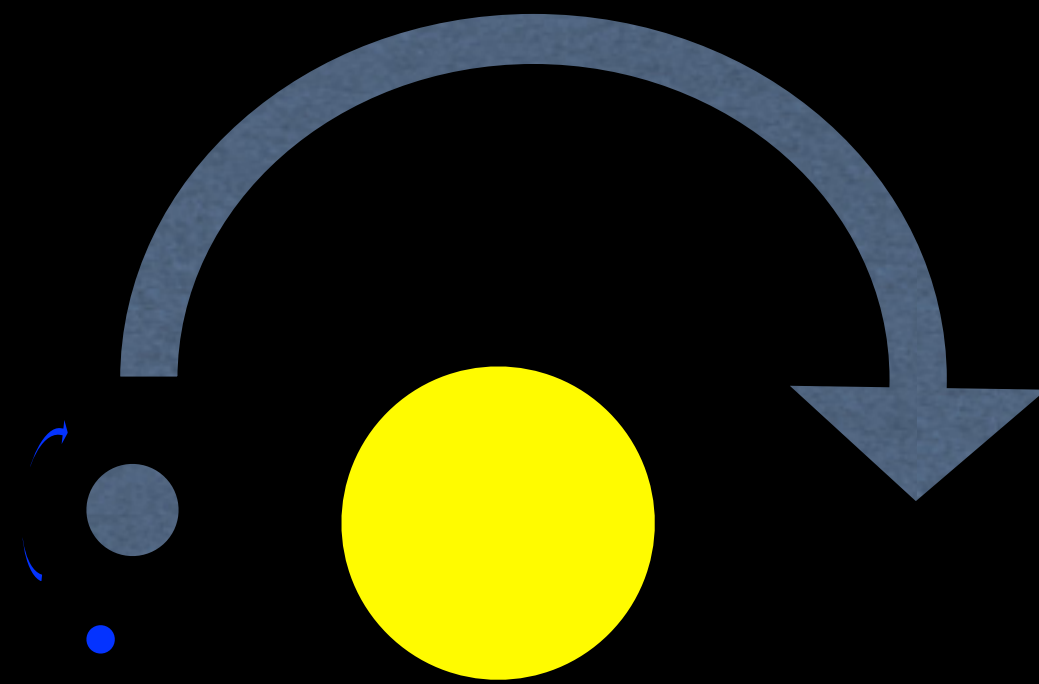
- Parent child relationship for motion components

- Hierarchical Modeling



Hierarchical Modeling

Relative motion → Parent-child relationship
Simplifies motion specification



Constrains motion → Reduces dimensionality

Modeling & Animating Hierarchies

3 Aspects

- Linkage & Joints - the relationships
- Data structure – how to represent such a hierarchy
- Converting local coordinate frames into global space

Terms

- Joint – To allow relative motion & parameters
 - Joint Limits – limit on valid joint angle values
 - Link/Segment – object involved in relative motion
 - Linkage – entire joint-link hierarchy
 - End effector – most distant link in linkage
 - Articulation variable – parameter of motion associated with joint
 - Pose – configuration of linkage using given set of joint angles
 - Pose vector – complete set of joint angles for linkage
-
- Arc of a tree data structure – corresponds to a joint
 - Node of a tree data structure – corresponds to a link



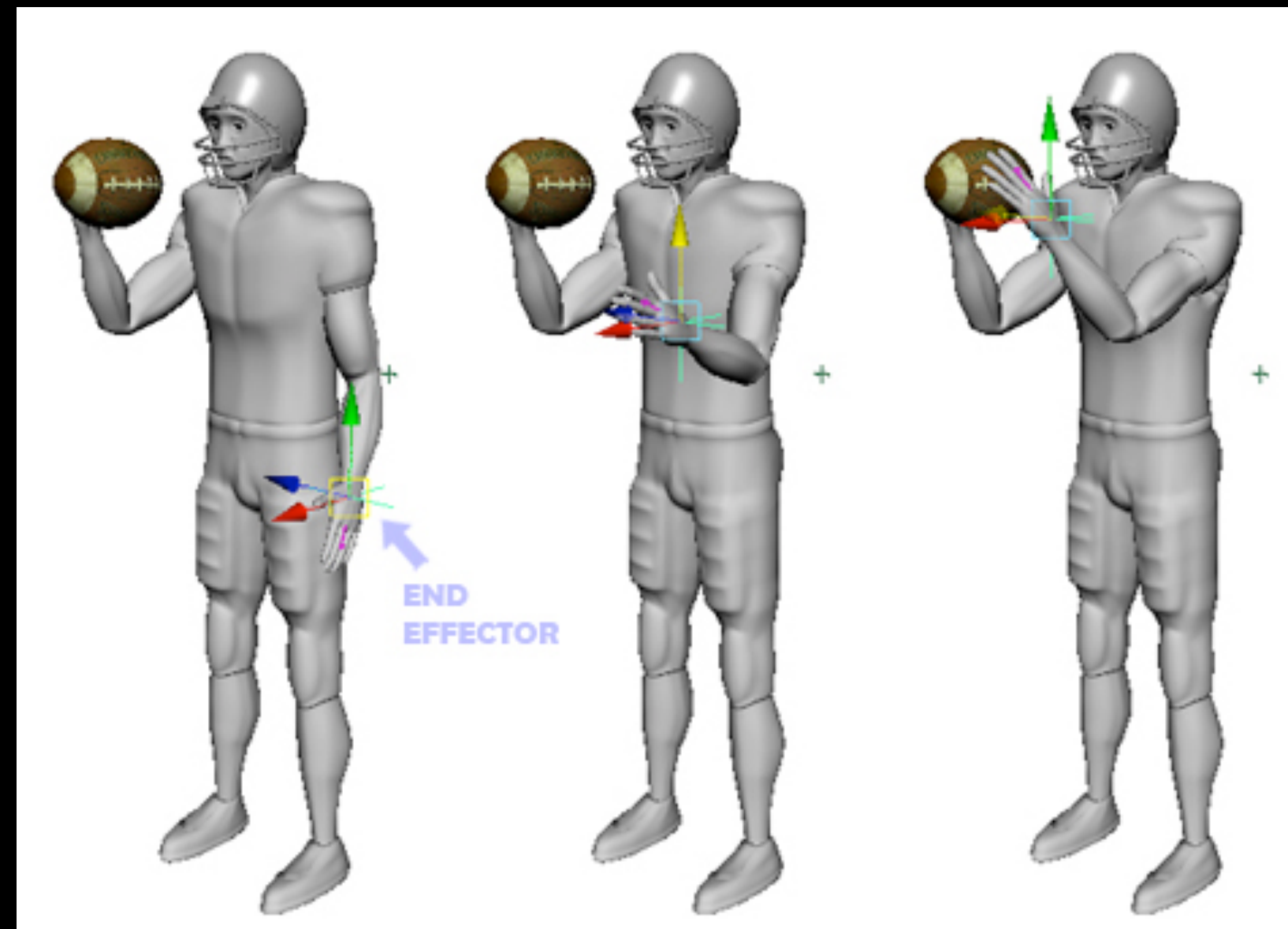
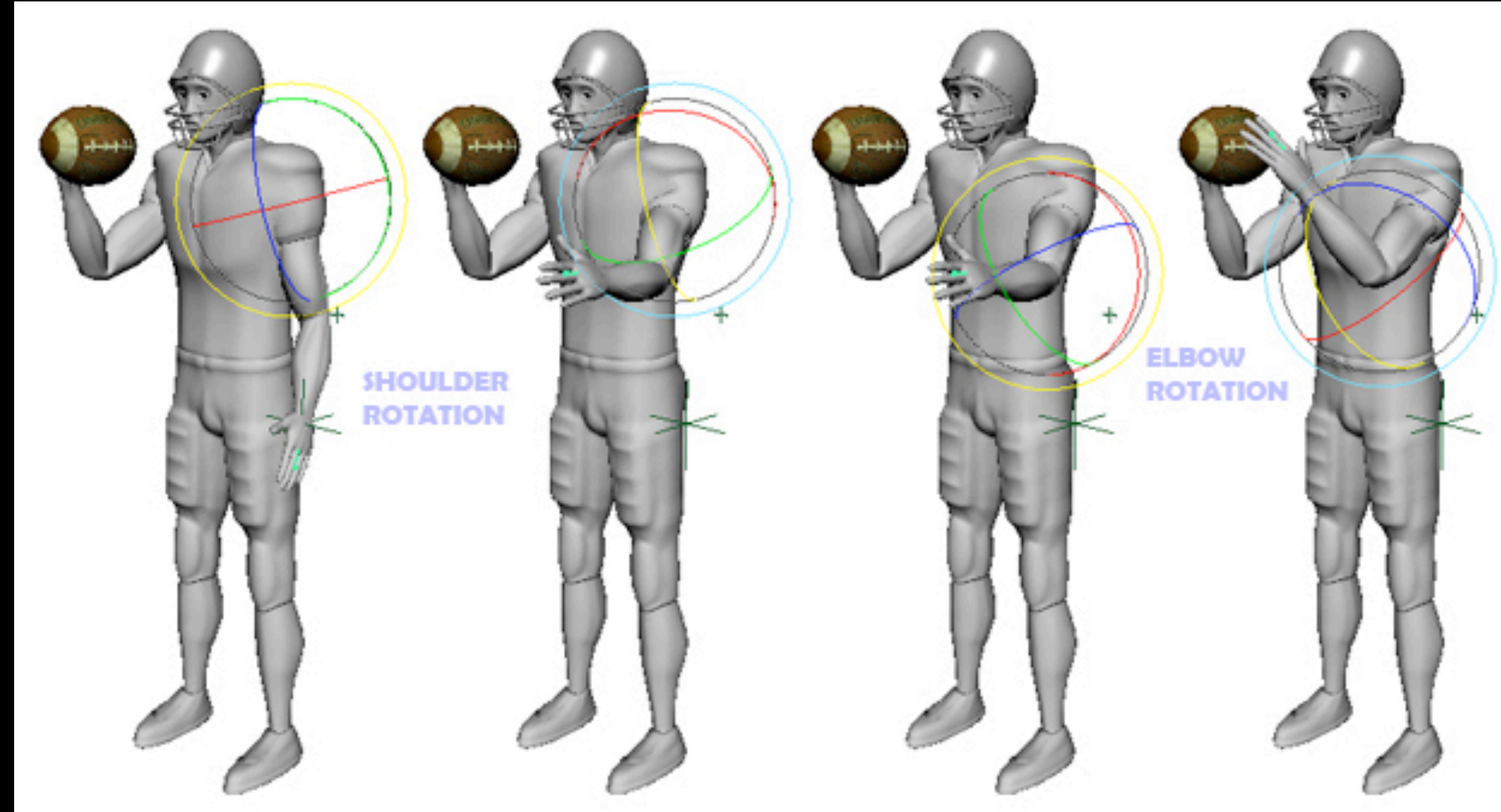
What to be Specified/Controlled

- Configuration of linkage
 - Mostly static
- Shape of linkage
 - Mostly static
- Joint movements!!!

Use of Hierarchies in Animation

- Forward Kinematics (FK)
 - animator specifies values of articulation variables
 - global transform for each linkage is computed
- Inverse Kinematics (IK)
 - animator specifies final desired global transform for end effector (and possibly other linkages)
 - Values of articulation variables are computed

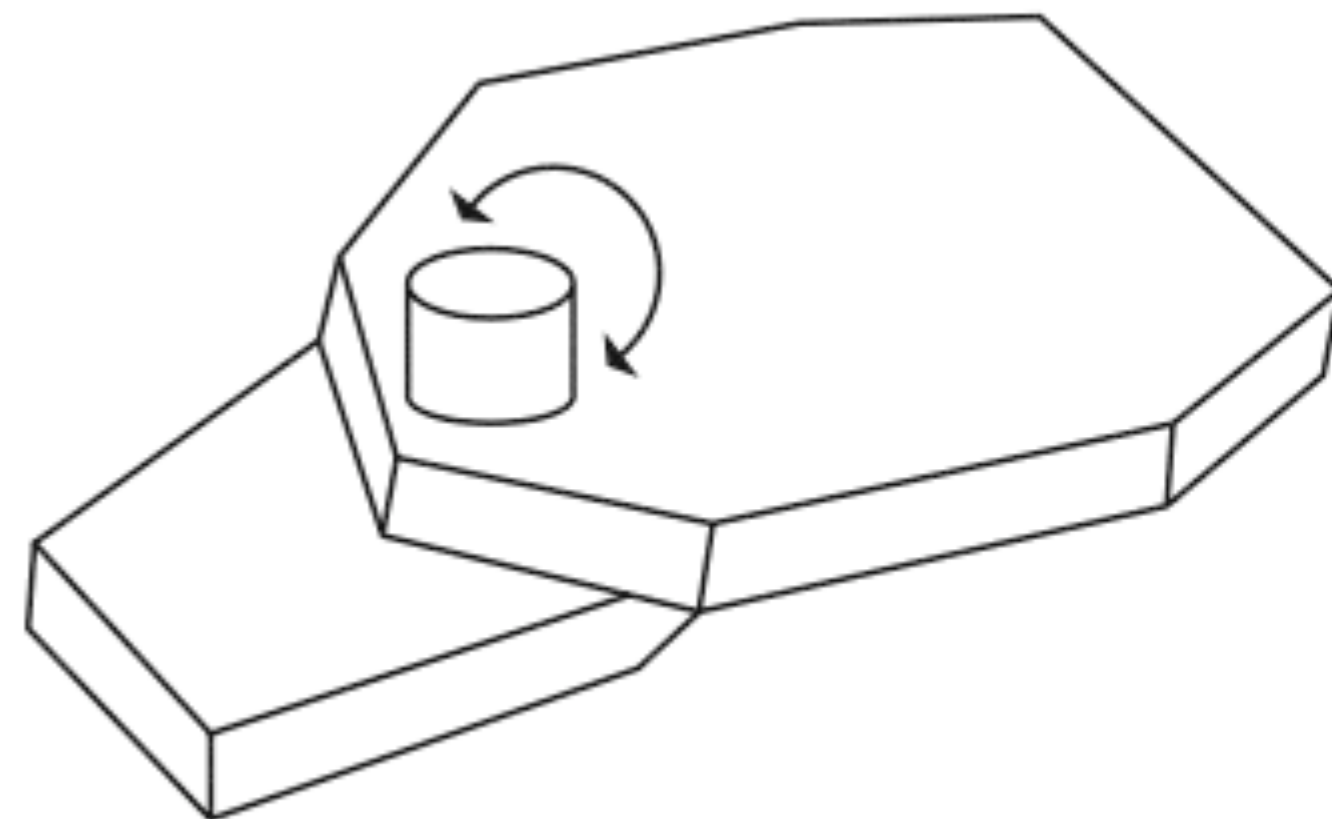
Forward & Inverse Kinematics



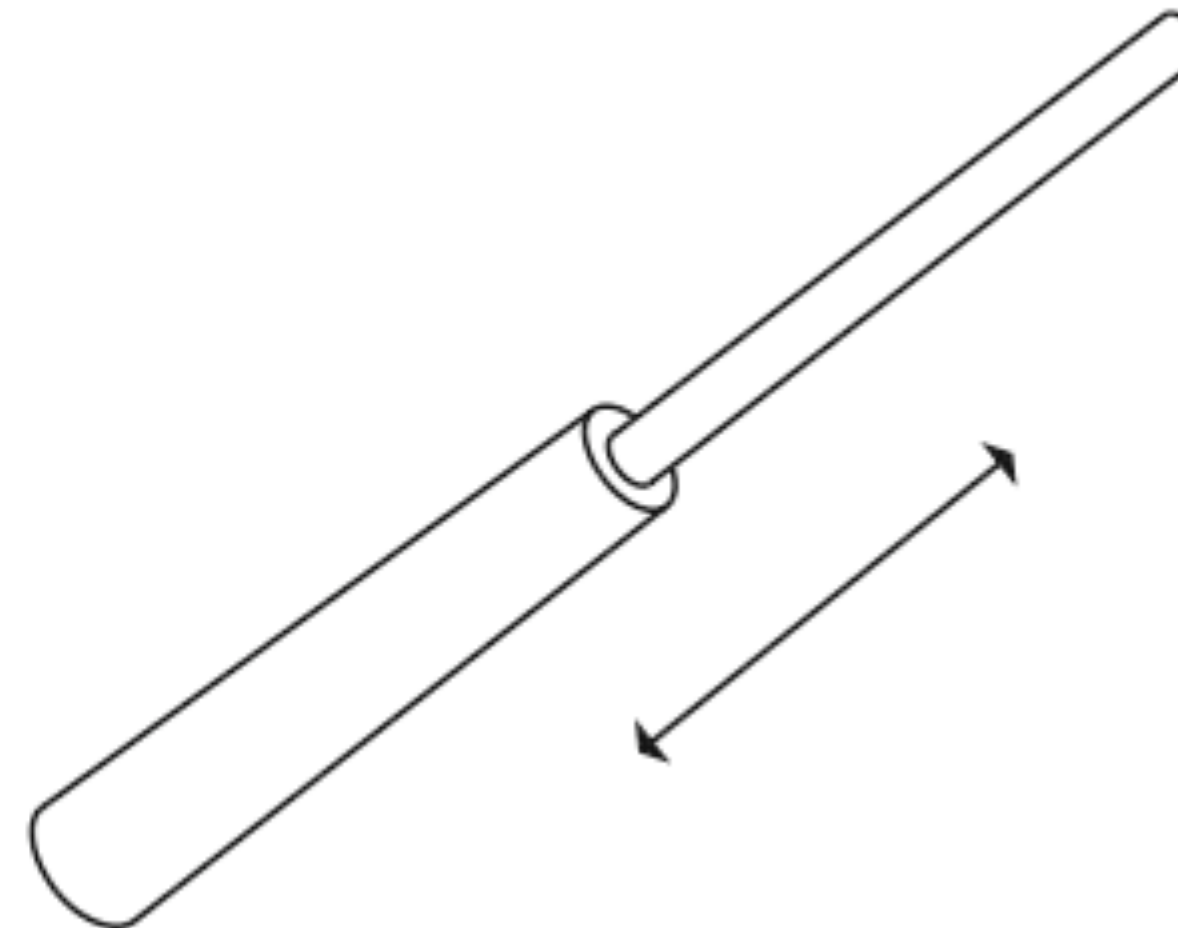
Controlling the movement

- Joint is the key
- What to control for each joints?
- How the joint look like?

Joints – relative movement

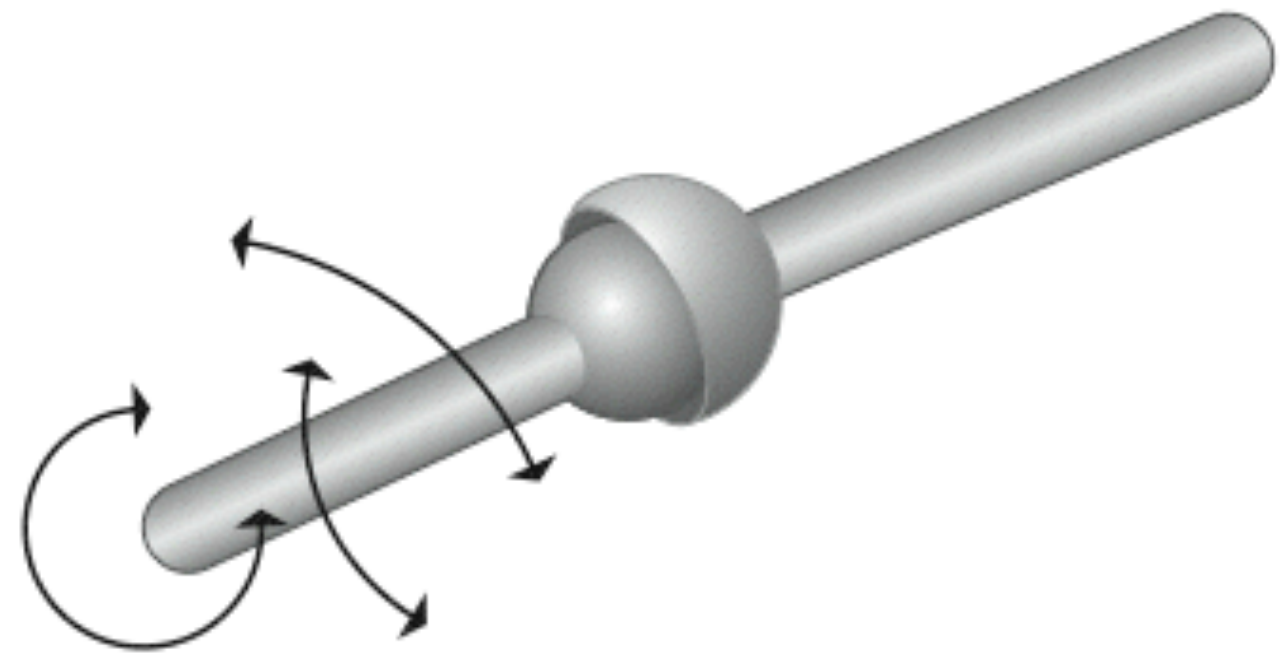


Revolute joint

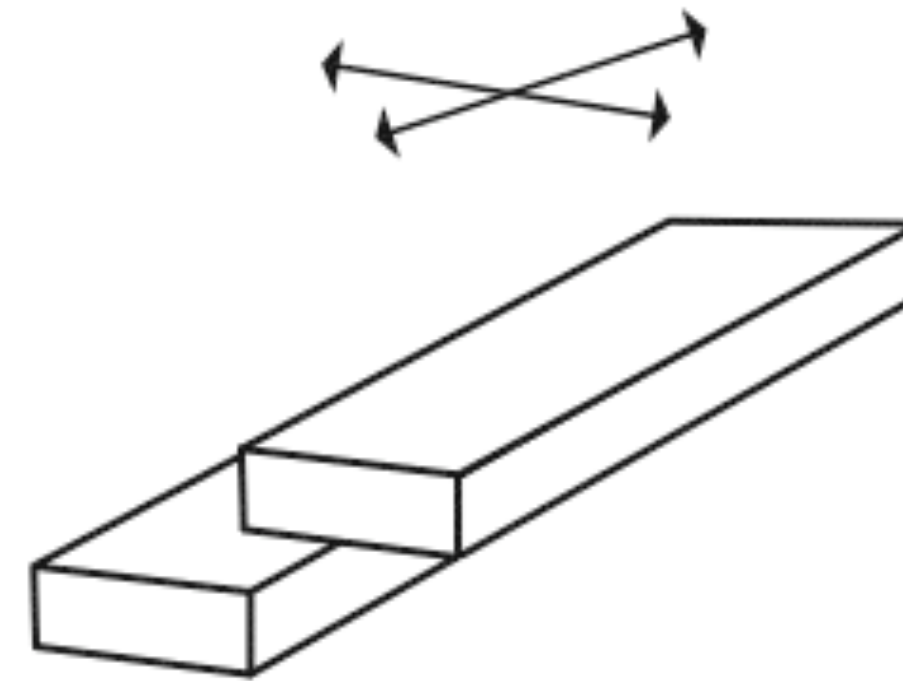


Prismatic joint

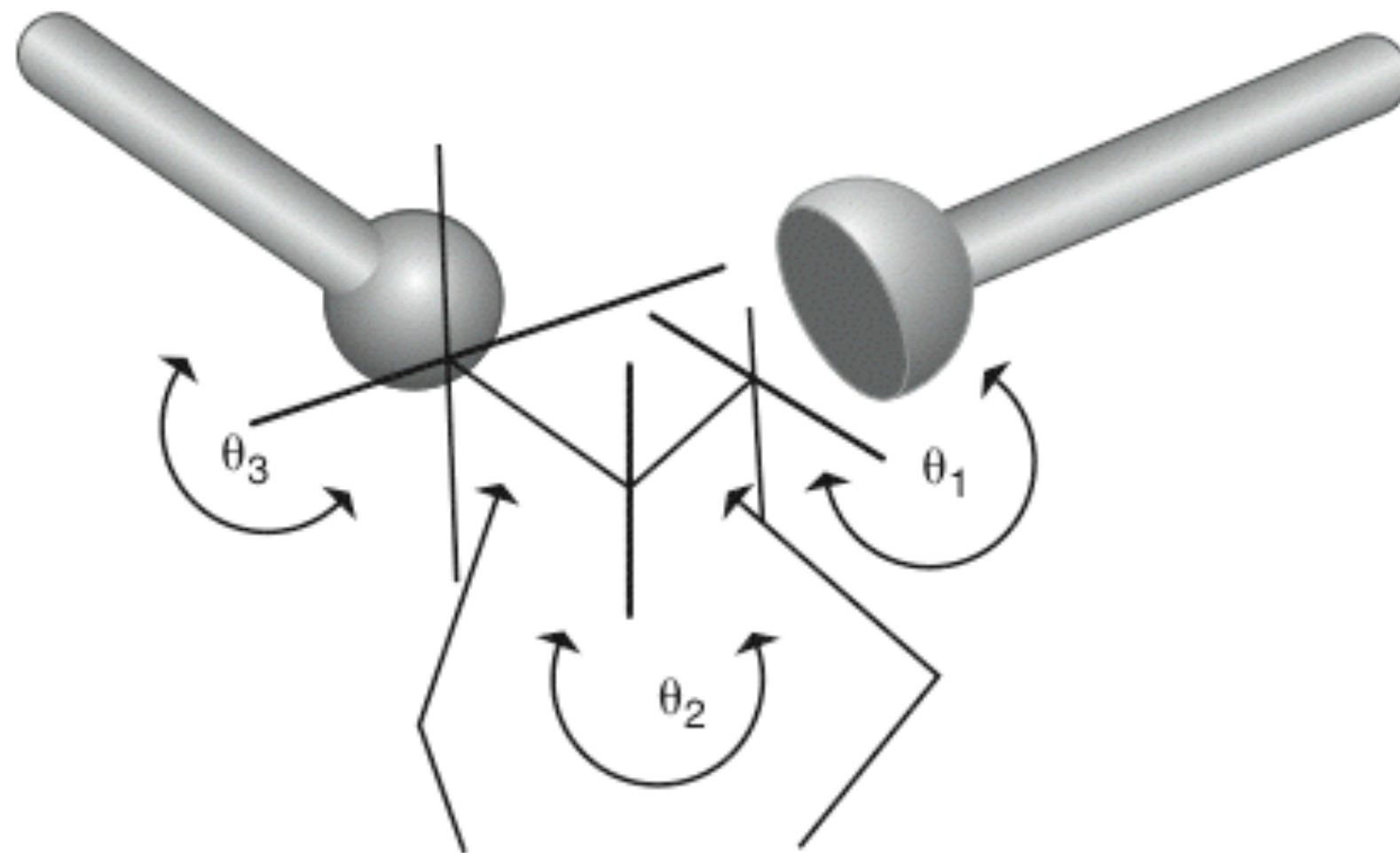
Complex Joints



Ball-and-socket joint

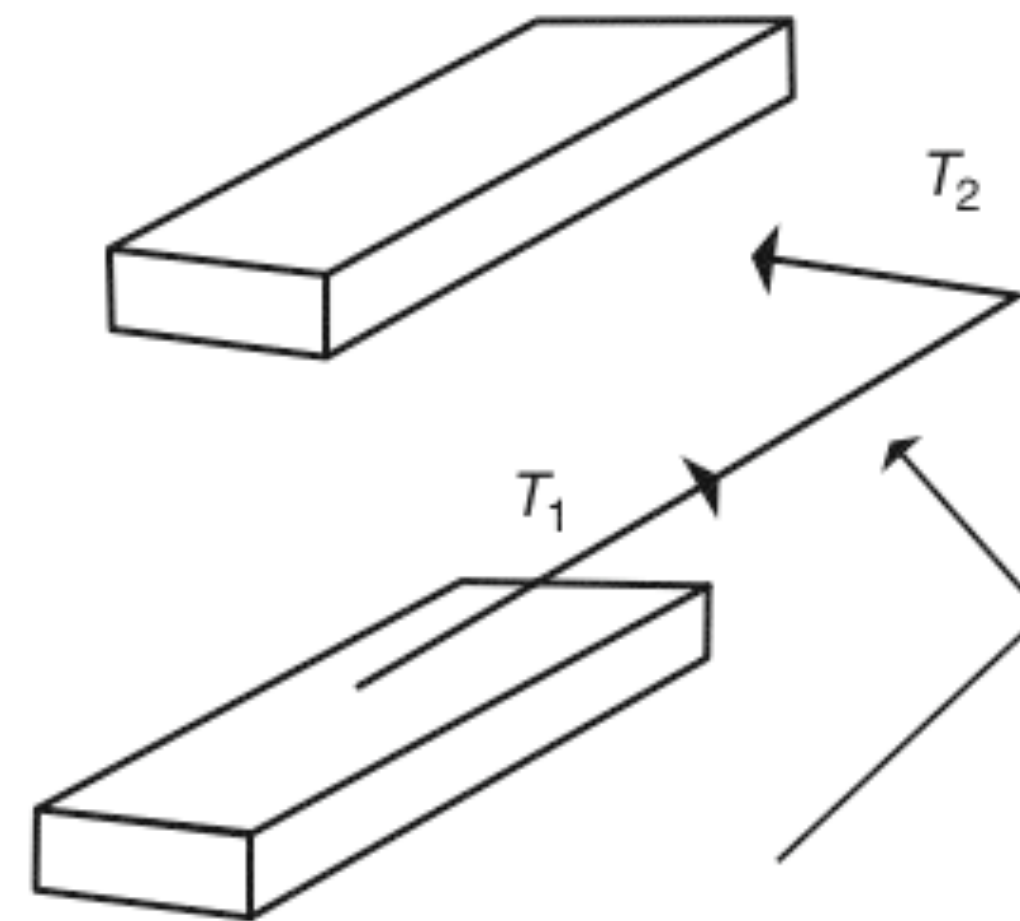


Planar joint



zero-length linkages

Ball-and-socket joint modeled as 3 one-degree joints with zero-length links

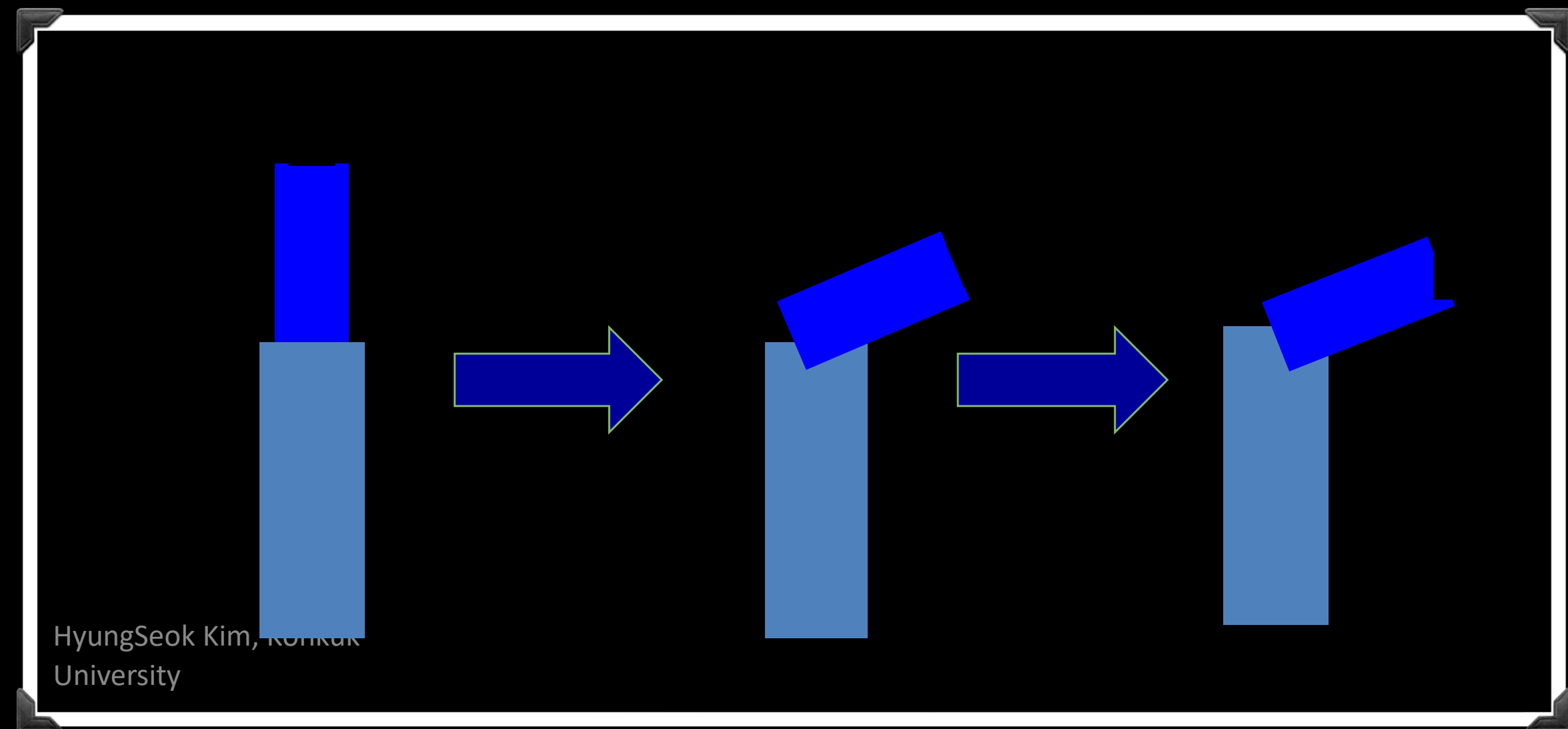


zero-length linkage

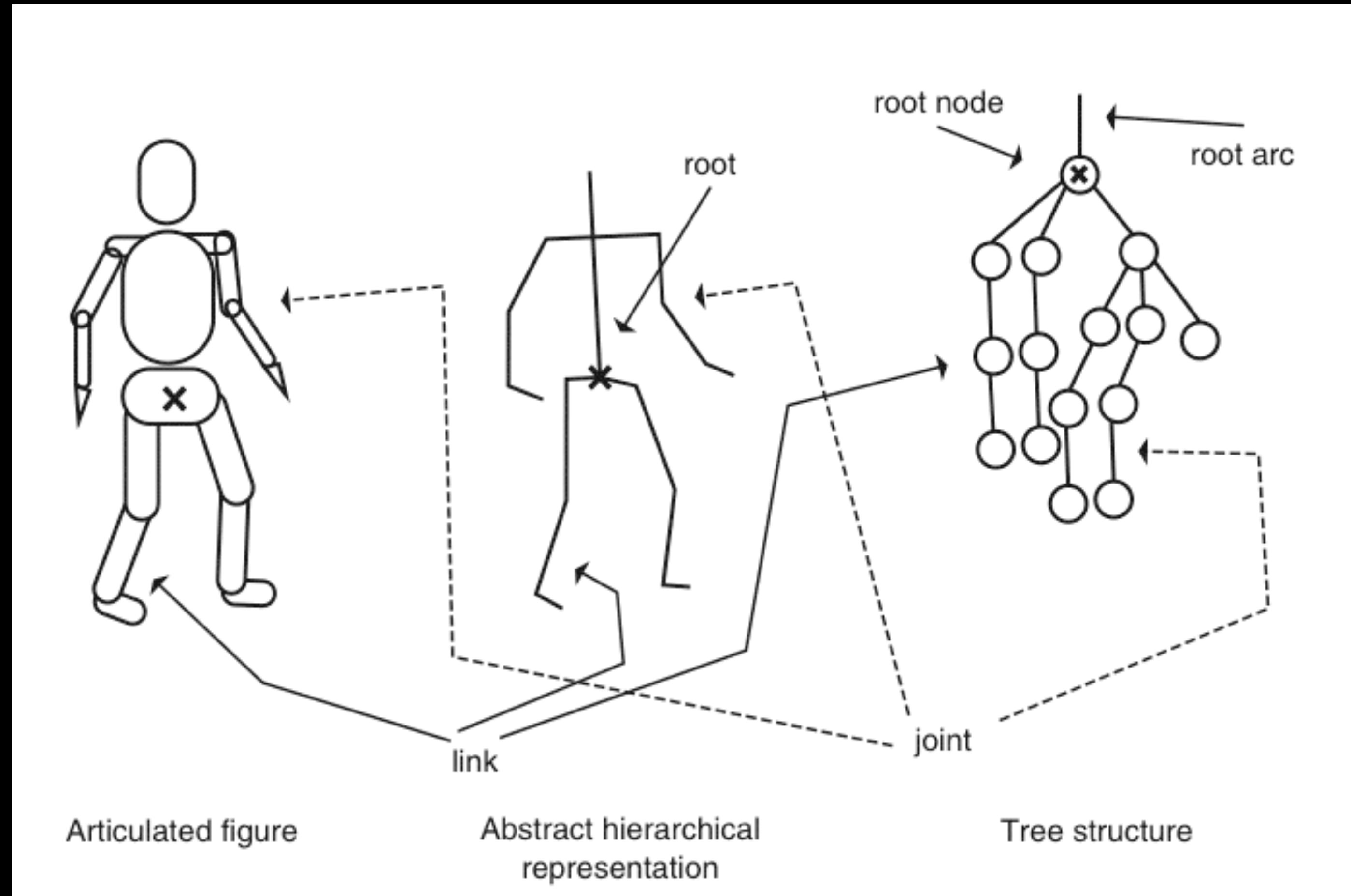
Planar joint modeled as 2 one-degree prismatic joints with zero-length links

Joint Movements

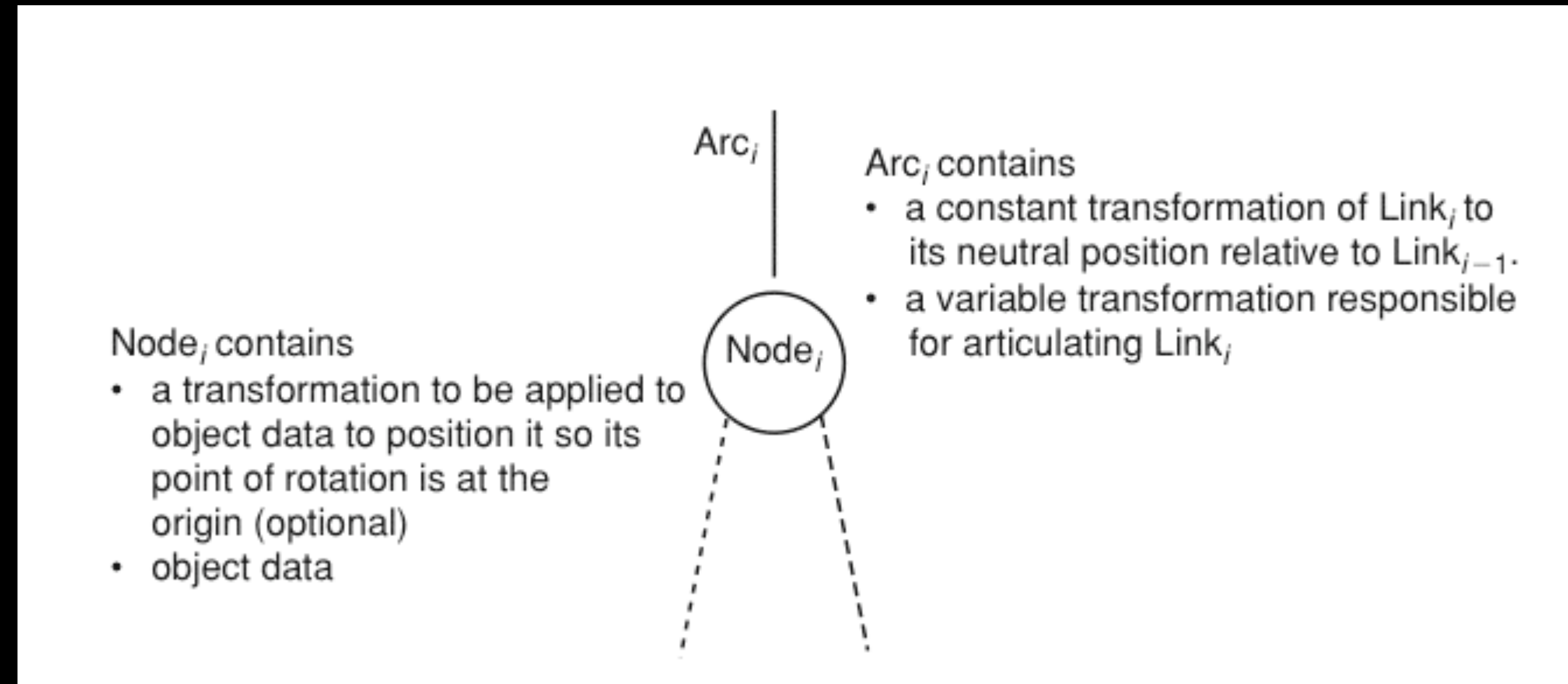
- Each joint have values
 - Position of each link could be defined by relative position from the parent joint
- Examples
- Global representation?



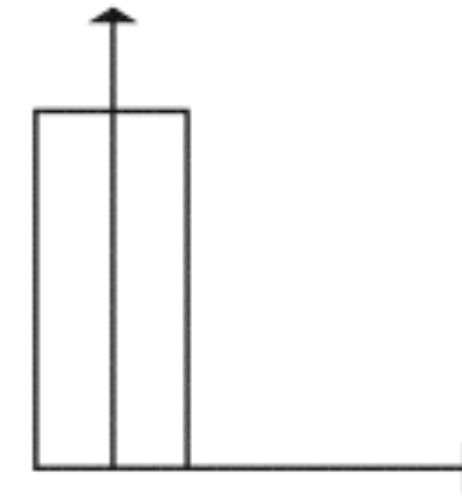
Hierarchical structure



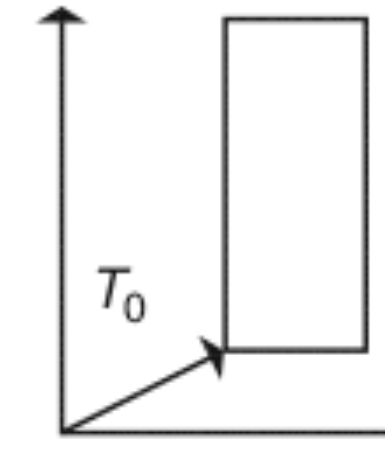
Tree structure



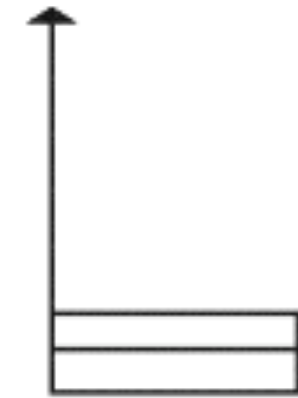
Tree structure



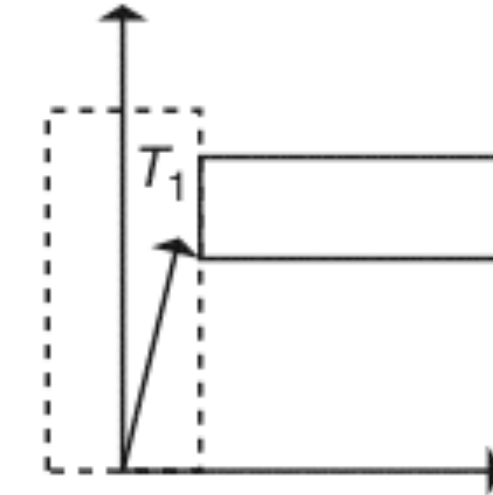
Original definition of root object ($Link_0$)



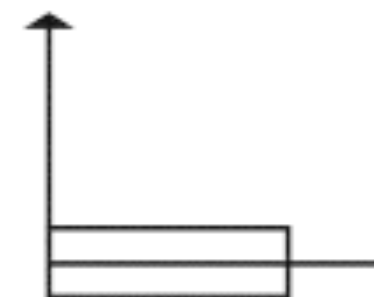
Root object ($Link_0$) transformed (translated and scaled) by T_0 to some known location in global space



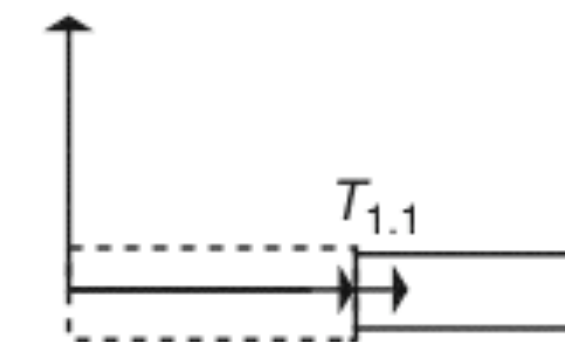
Original definition of $Link_1$



$Link_1$ transformed by T_1 to its position relative to untransformed $Link_0$

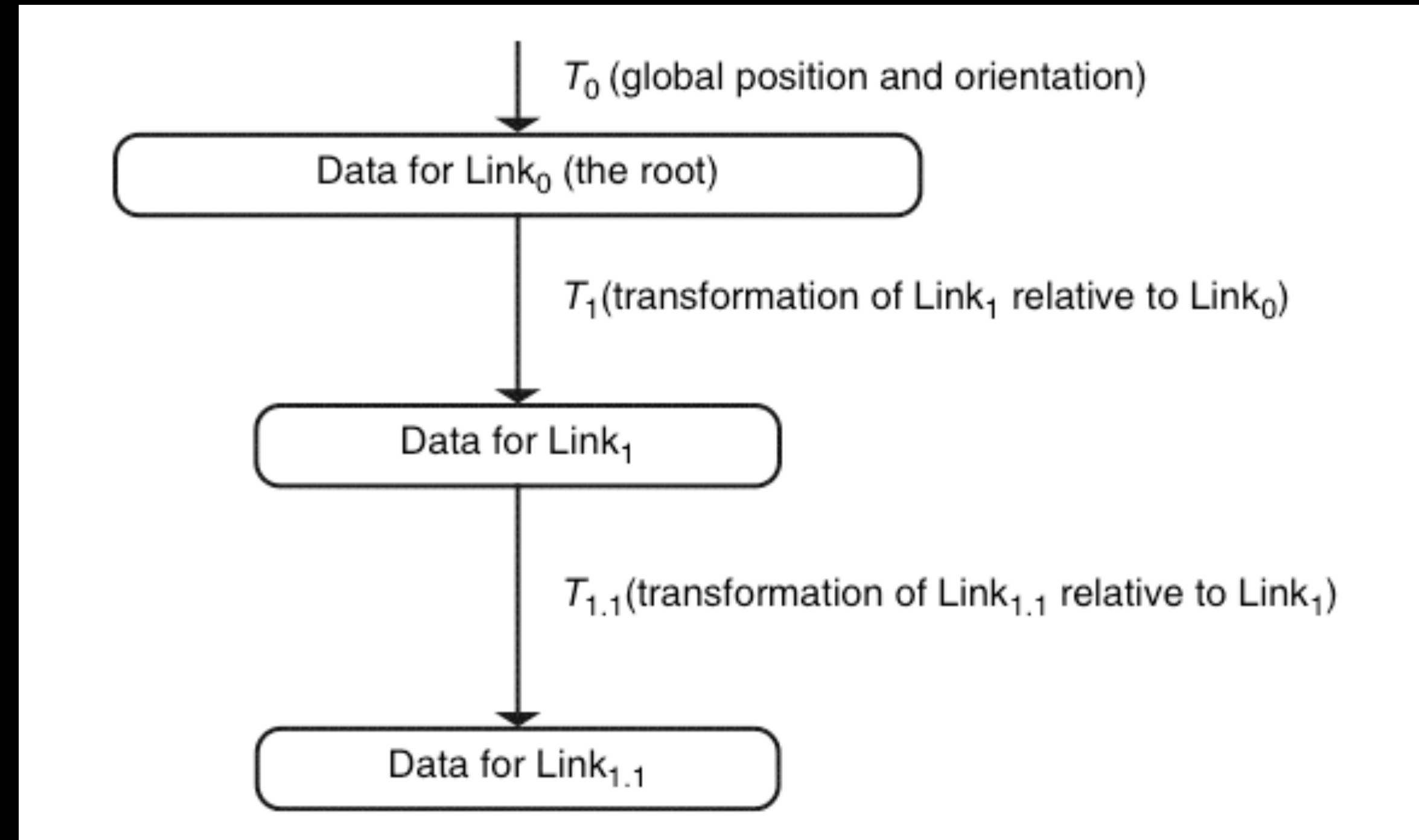


Original definition of $Link_{1,1}$

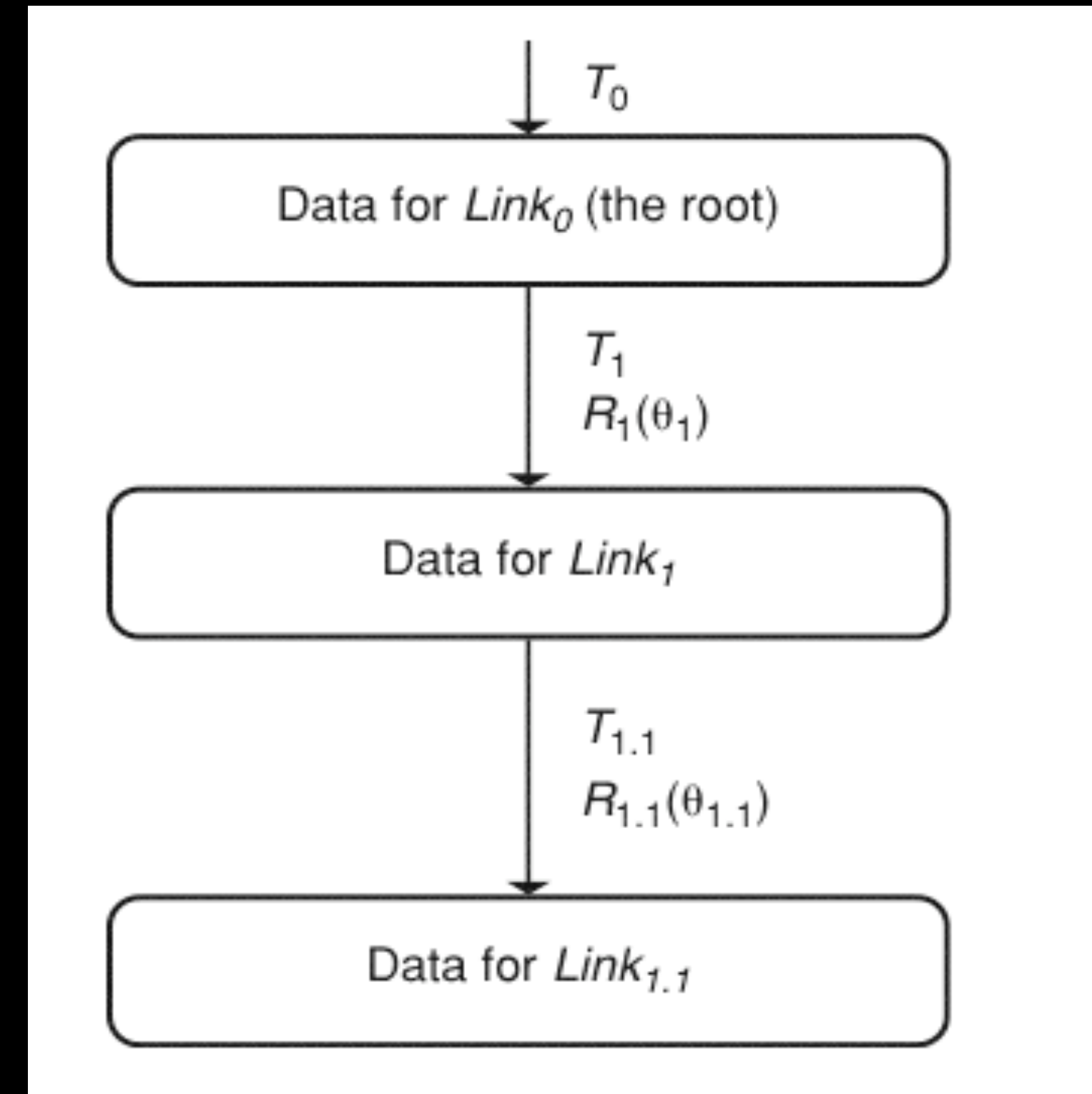
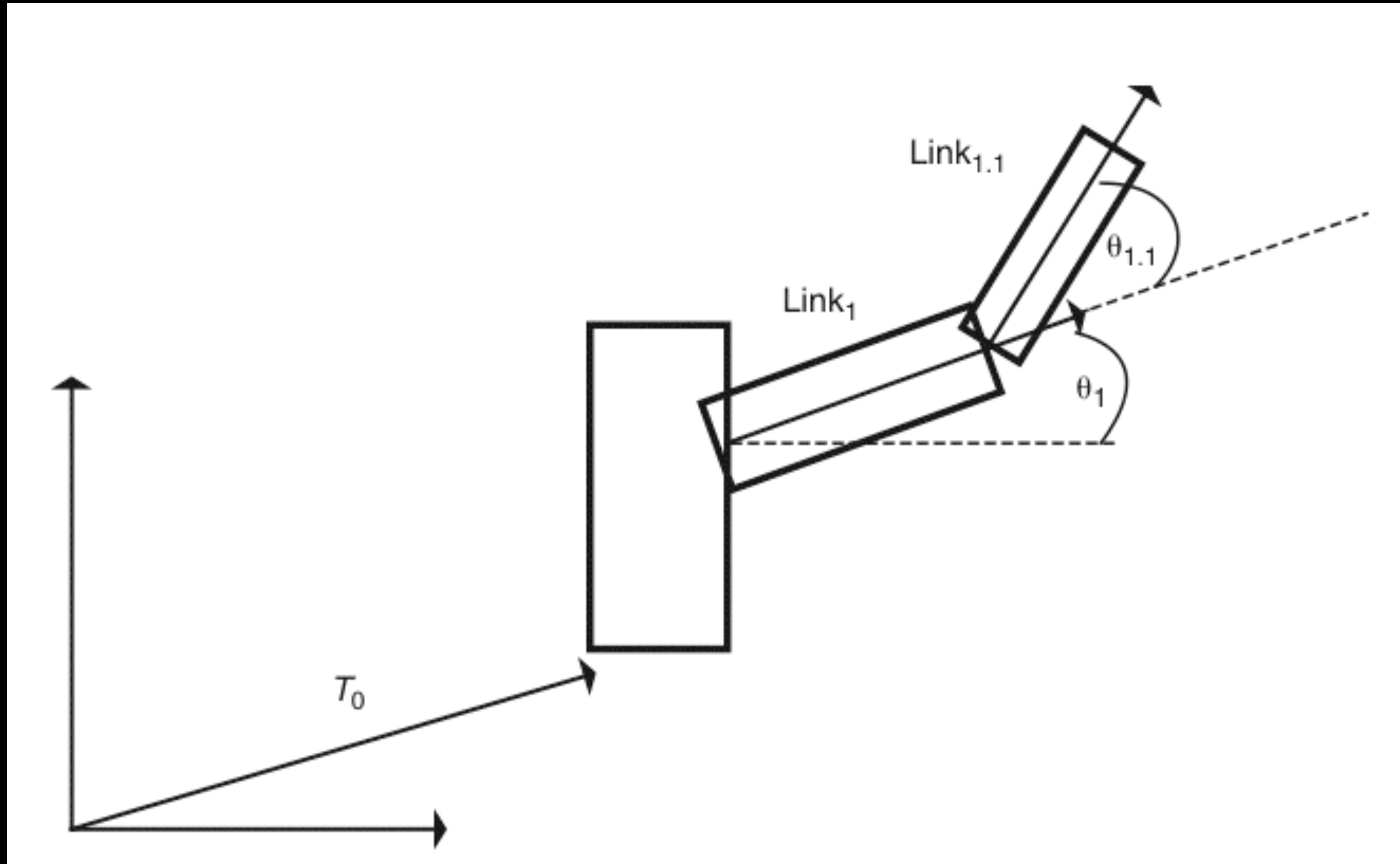


$Link_{1,1}$ transformed by $T_{1,1}$ to its position relative to untransformed $Link_1$

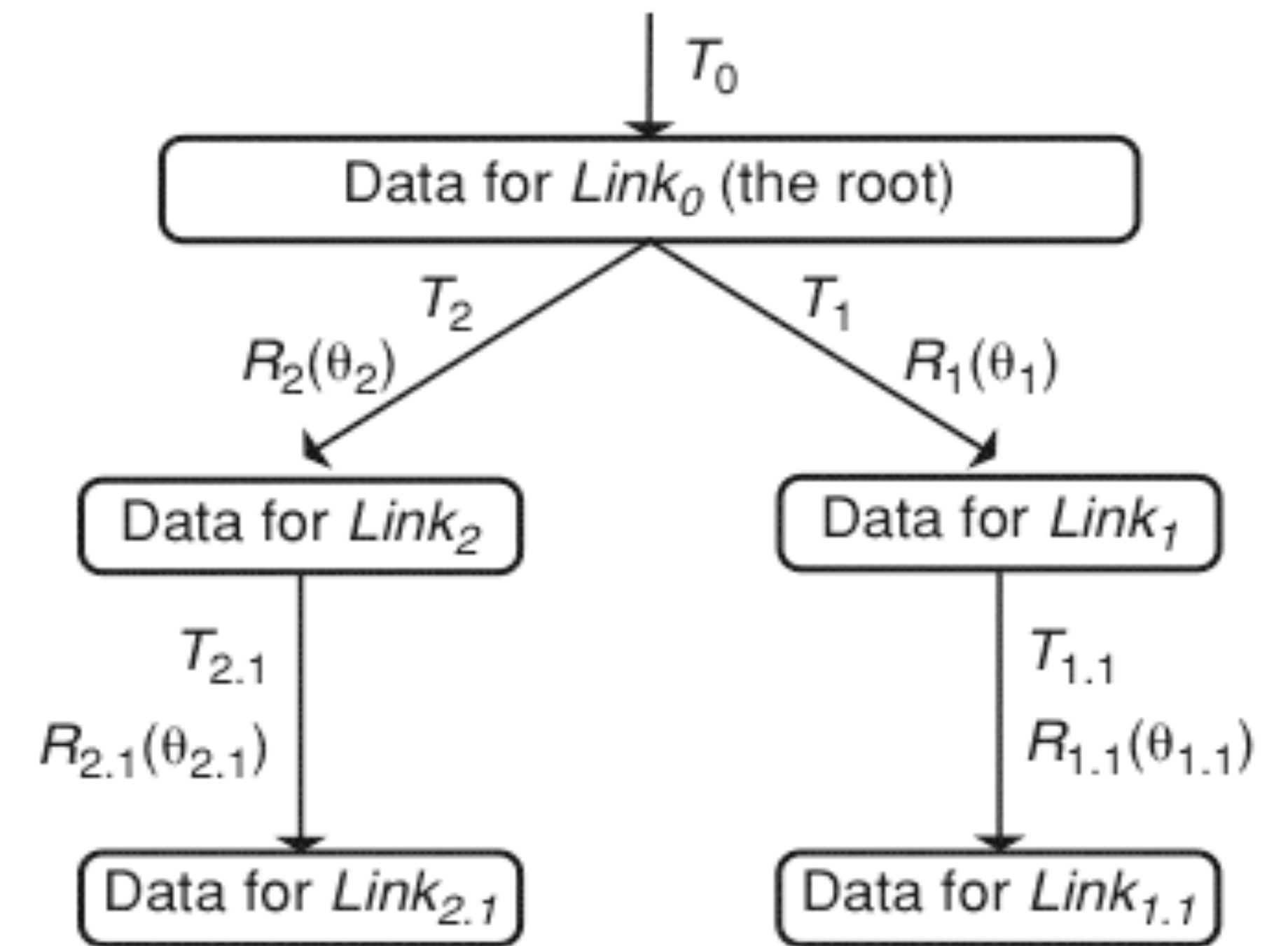
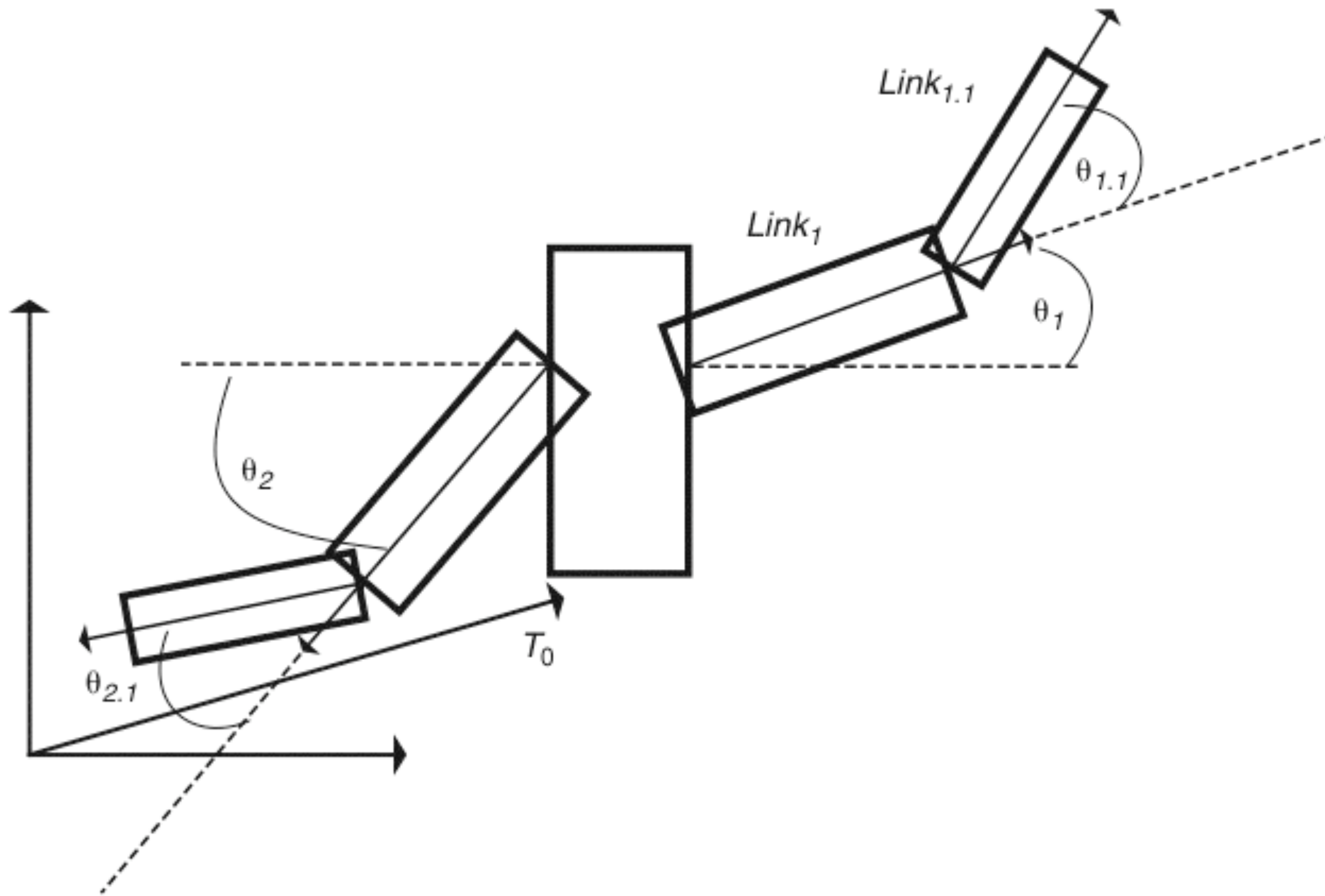
Tree structure



Relative movement



Tree structure

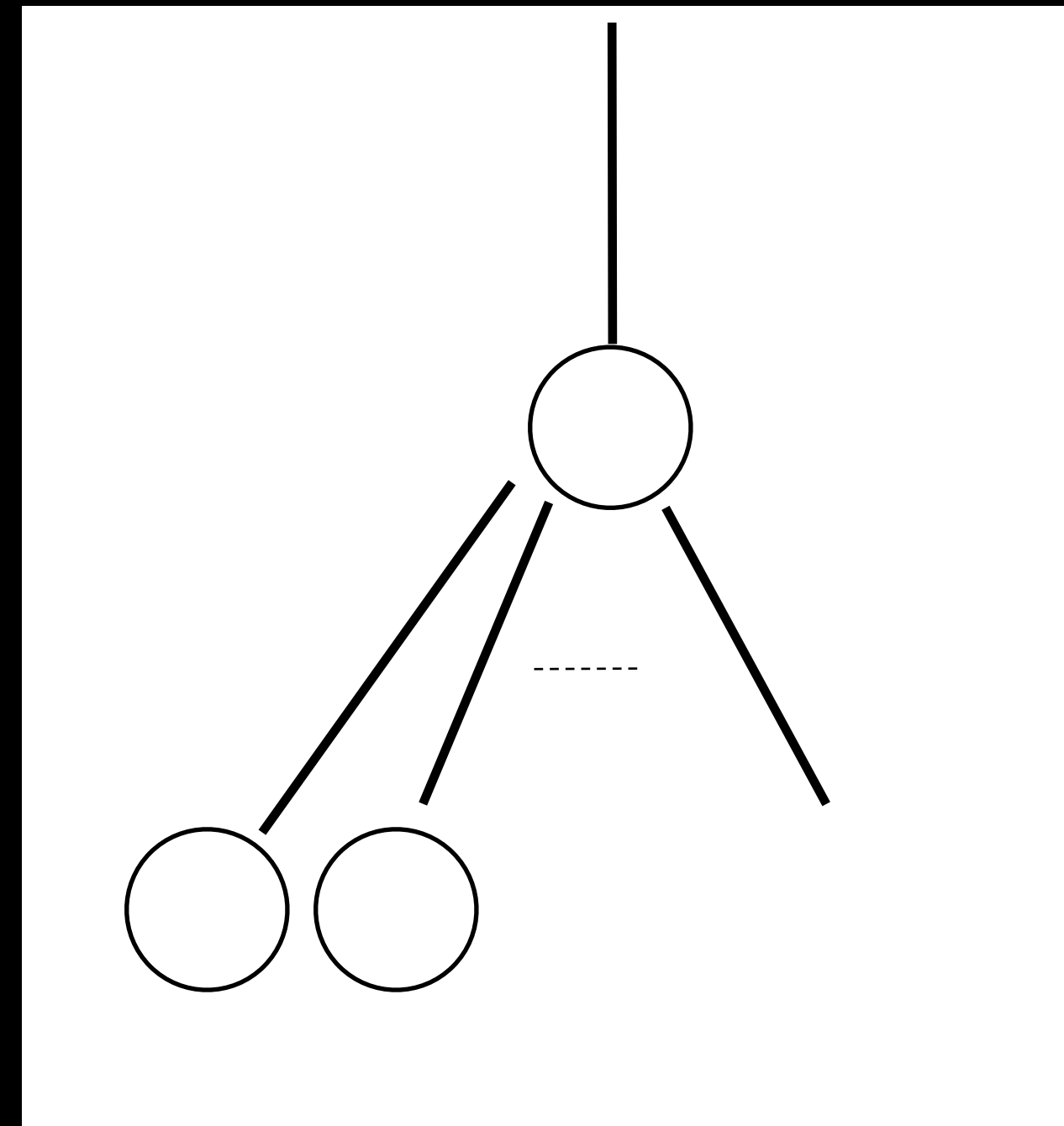


Implementation note

Nodes & arcs

NODE

Pointer to data
Data transformation
Pointer to arcs



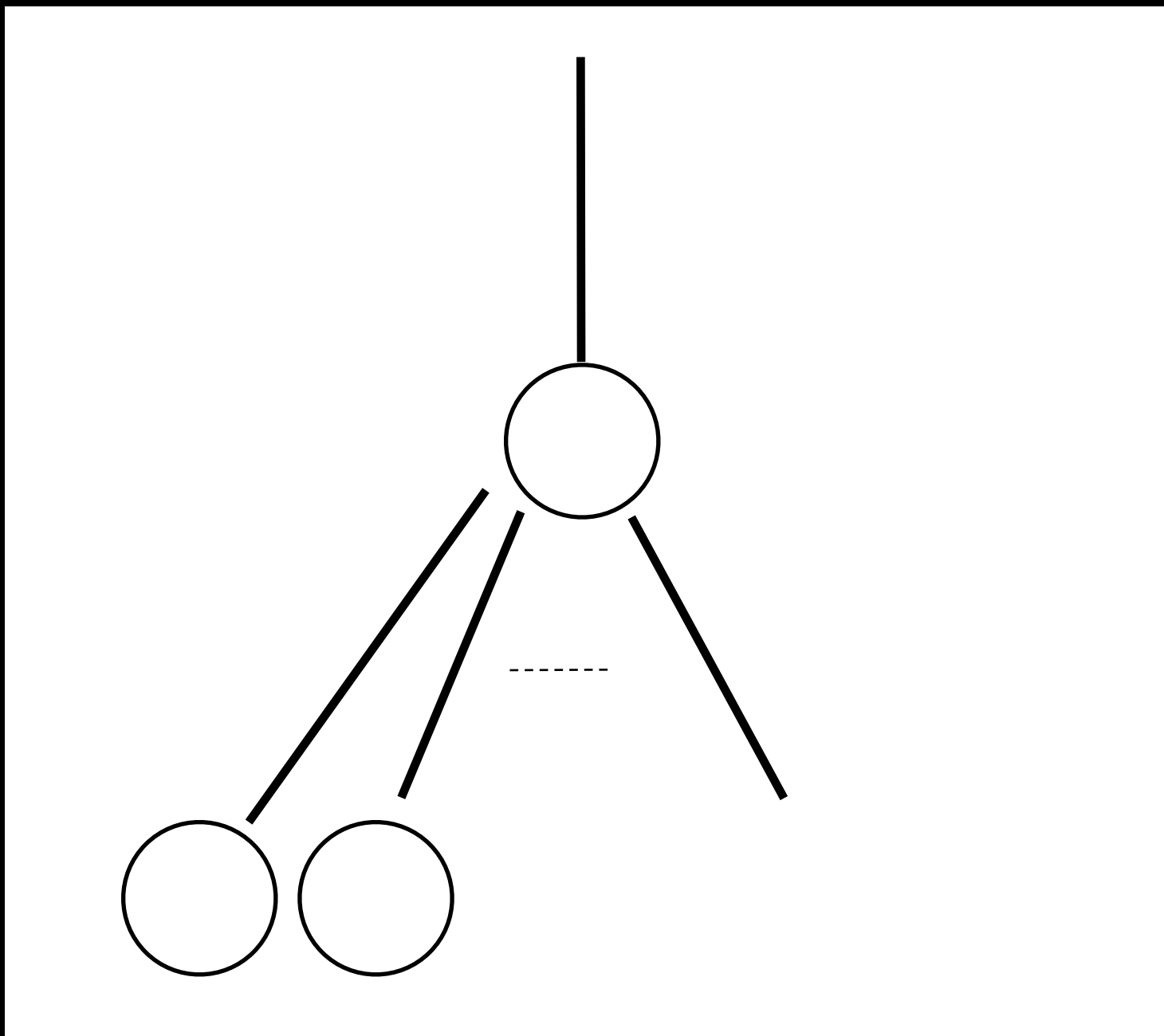
ARC

Transform of one next node relative to parent node
Articulation transform
Pointer to node

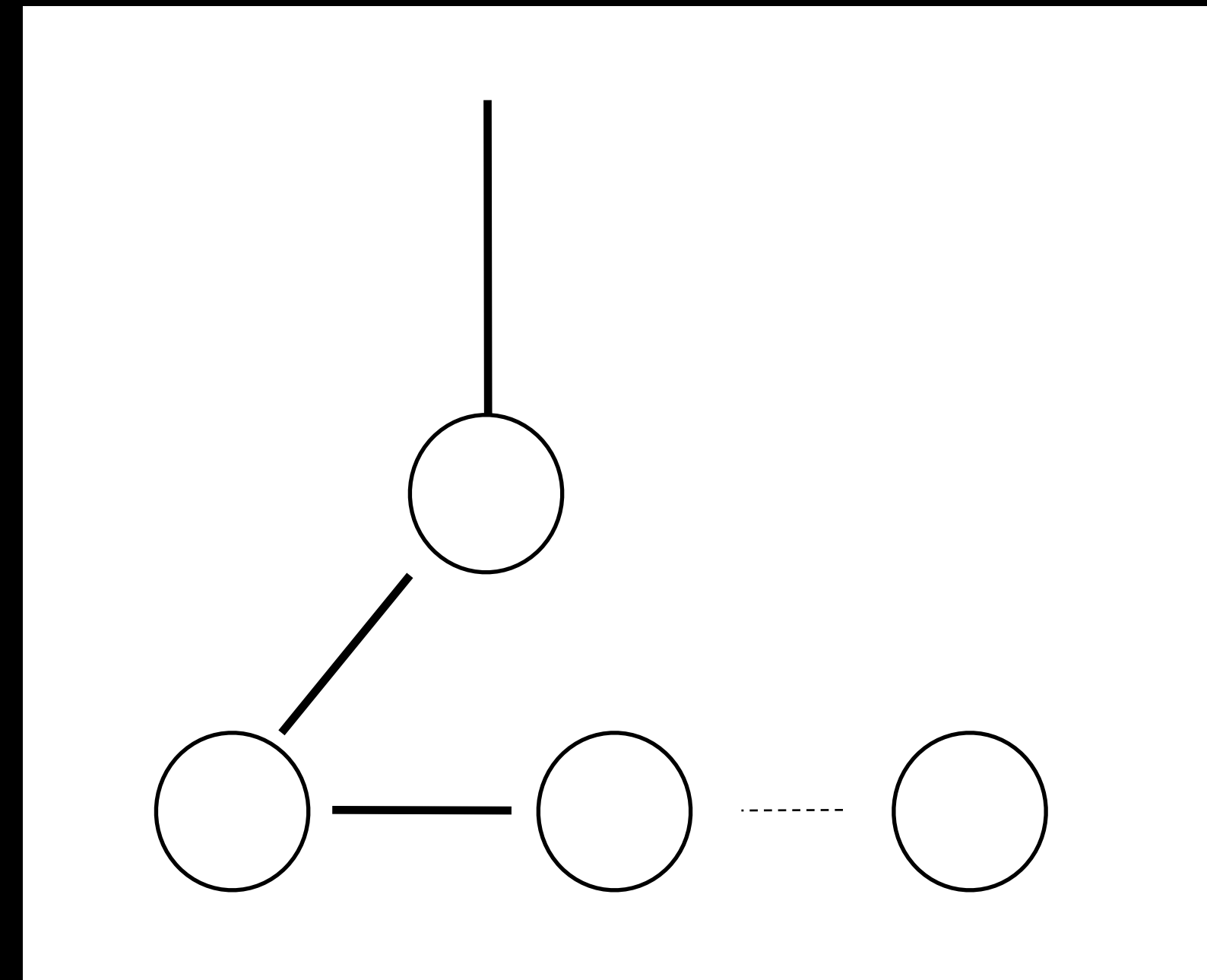
Implementation note

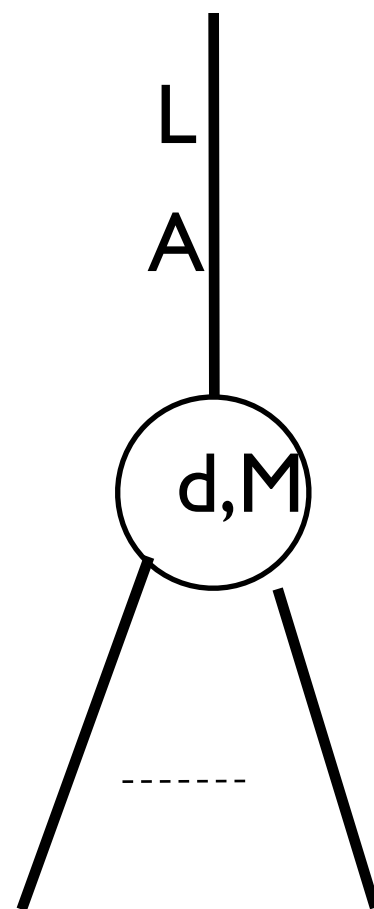
Representing arbitrary number of children with fixed-length data structure

Use array of pointers to
children
In node, arcPtr[]



Node points to first child
Each child points to sibling
Last sibling points to NULL
In node: arcPtr for 1st child
In arc: arcPtr for sibling





```
traverse (arcPtr,matrix)
{
    // concatenate arc matrices
    matrix = matrix*arcPtr->Lmatrix
    matrix = matrix*arcPtr->Amatrix;

    // get node and transform data
    nodePtr=acrPtr->nodePtr
    push (matrix)
    matrix = matrix * nodePTr->matrix
    aData = transformData(matrix,dataPTr)
    draw(aData)
    matrix = pop();

    // process children
    If (nodePtr->arcPtr != NULL) {
        nextArcPtr = nodePTr-> arcPtr
        while (nextArcPtr != NULL) {
            push(matrix)
            traverse(nextArcPtr,matrix)
            matrix = pop()
            nextArcPtr = nextArcPtr->arcPtr
        }
    }
}
```

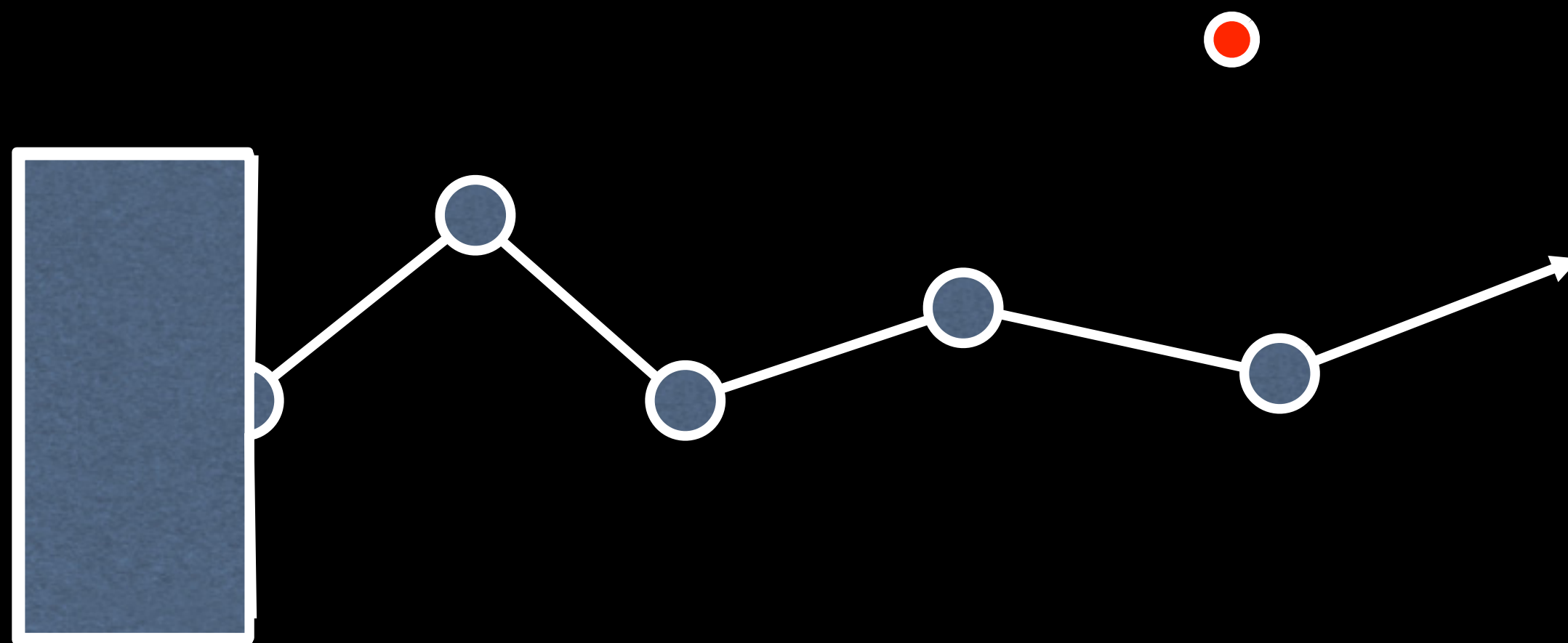
Inverse kinematics

Given goal position (and orientation) for end effector

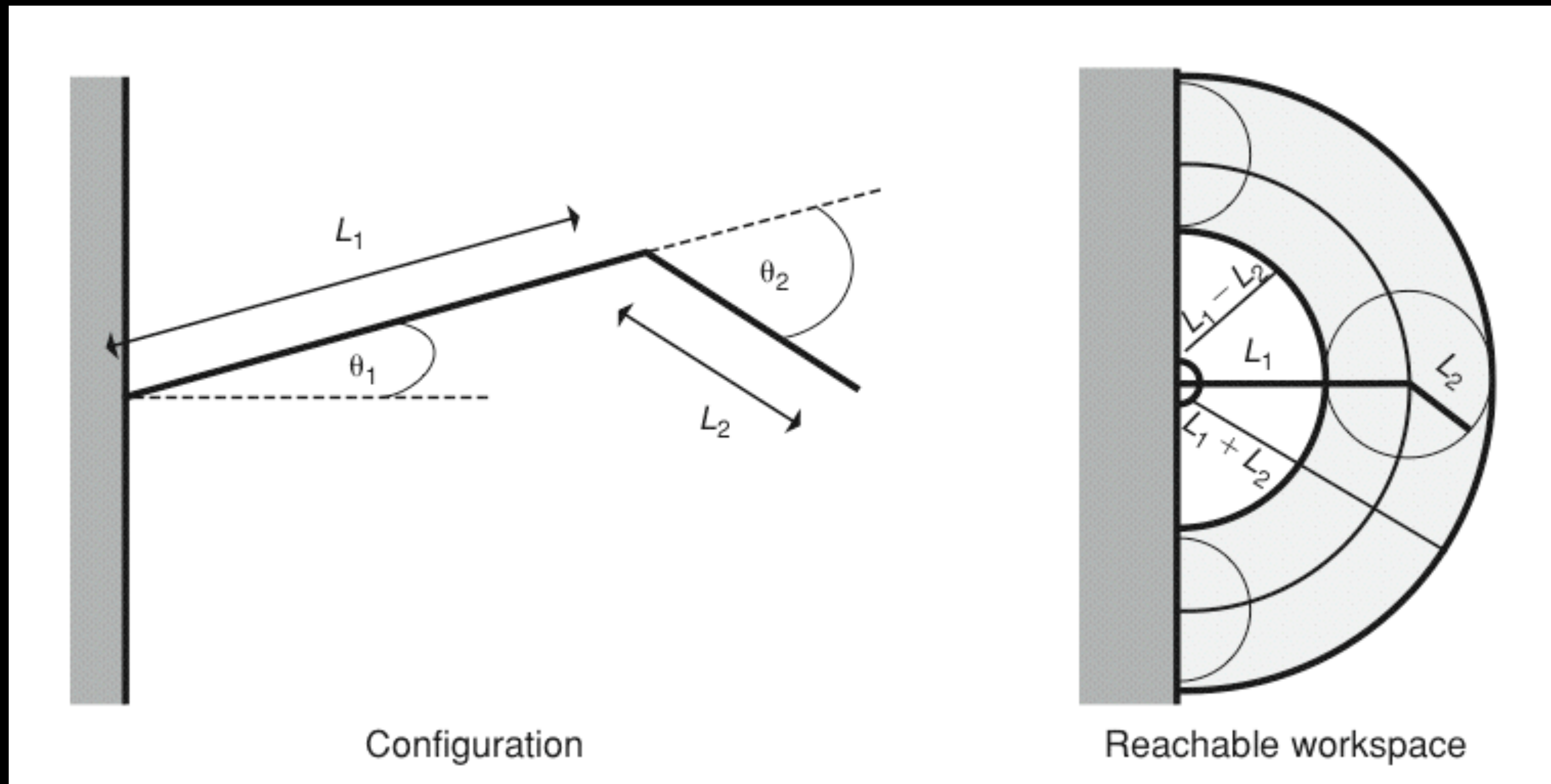
Compute internal joint angles

If simple enough $\Rightarrow$ analytic solution

Else $\Rightarrow$ numeric iterative solution

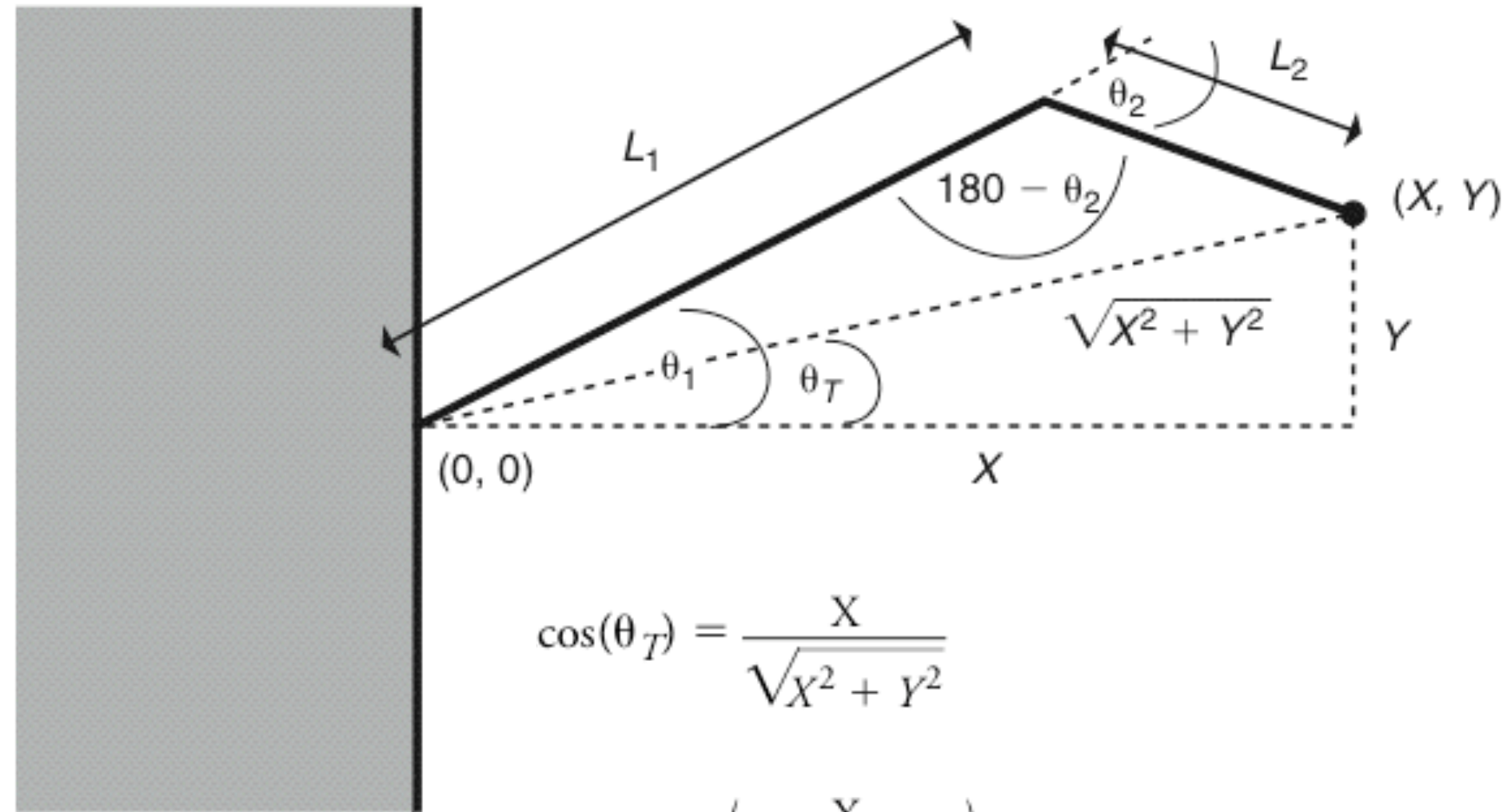


Inverse kinematics - spaces



Configuration space
Reachable workspace
Dextrous workspace

Analytic inverse kinematics



$$\cos(\theta_T) = \frac{X}{\sqrt{X^2 + Y^2}}$$

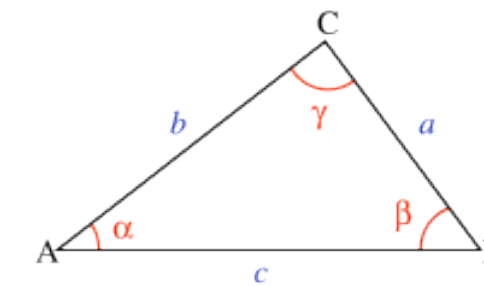
$$\theta_T = \arccos\left(\frac{X}{\sqrt{X^2 + Y^2}}\right)$$

$$\cos(\theta_1 - \theta_T) = \frac{L_1^2 + X^2 + Y^2 - L_2^2}{2L_1\sqrt{X^2 + Y^2}}$$

$$\theta_1 = \arccos\left(\frac{L_1^2 + X^2 + Y^2 - L_2^2}{2L_1\sqrt{X^2 + Y^2}}\right) + \theta_T$$

$$\cos(180 - \theta_2) = -\cos(\theta_2) = \frac{L_1^2 + L_2^2 - (X^2 + Y^2)}{2L_1L_2}$$

$$\theta_2 = \arccos\left(-\frac{L_1^2 + L_2^2 - (X^2 + Y^2)}{2L_1L_2}\right)$$



$$c^2 = a^2 + b^2 - 2ab \cos \gamma$$

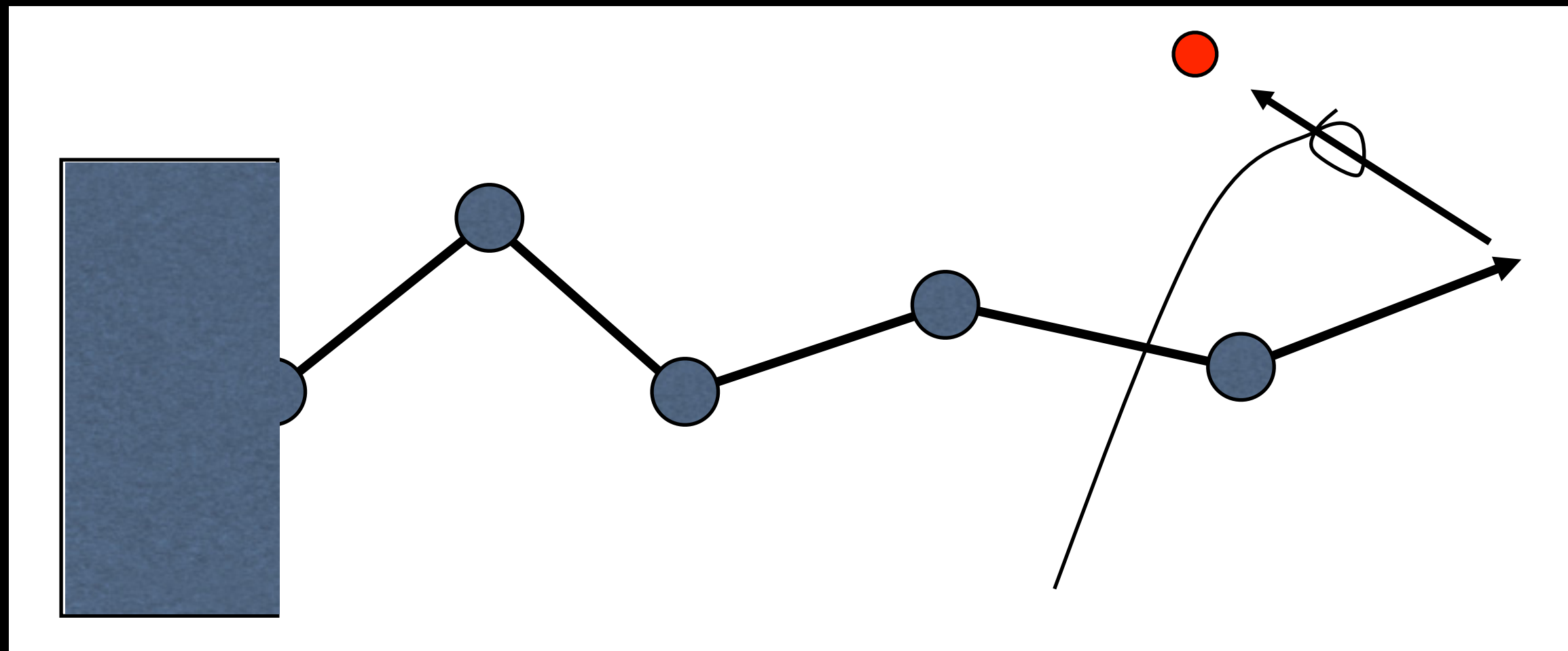
(cosine rule)

(cosine rule)

Inverse Kinematics - Numeric

If linkage is too complex to solve analytically

- E.g., human arm is typically modeled as 3-1-3 or 3-2-2 linkage
- Solve iteratively – numerically solve for step toward goal



- Desired change from this specific pose
- Compute set of changes to the pose to effect that change

Generic math notation

$$y_1 = f_1(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$y_2 = f_2(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$y_3 = f_3(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$y_4 = f_4(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$y_5 = f_5(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$y_6 = f_6(x_1, x_2, x_3, x_4, x_5, x_6)$$

$$Y = F(X)$$

Generic math notation

$$\frac{dy_i}{dt} = \frac{\partial f_i}{\partial x_1} \frac{dx_1}{dt} + \frac{\partial f_i}{\partial x_2} \frac{dx_2}{dt} + \frac{\partial f_i}{\partial x_3} \frac{dx_3}{dt} + \frac{\partial f_i}{\partial x_4} \frac{dx_4}{dt} + \frac{\partial f_i}{\partial x_5} \frac{dx_5}{dt} + \frac{\partial f_i}{\partial x_6} \frac{dx_6}{dt}$$

$$\dot{Y} = \frac{\partial F}{\partial X} \dot{X}$$

Inverse Kinematics - Jacobian

$$\dot{Y} = \frac{\partial F}{\partial X} \dot{X}$$

$$V = J(\theta) \dot{\theta}$$

Desired motion
of end effector



Unknown change in
articulation variables



The *Jacobian* is the matrix relating
the two: it's a function of current
variable values

Inverse Kinematics - Jacobian

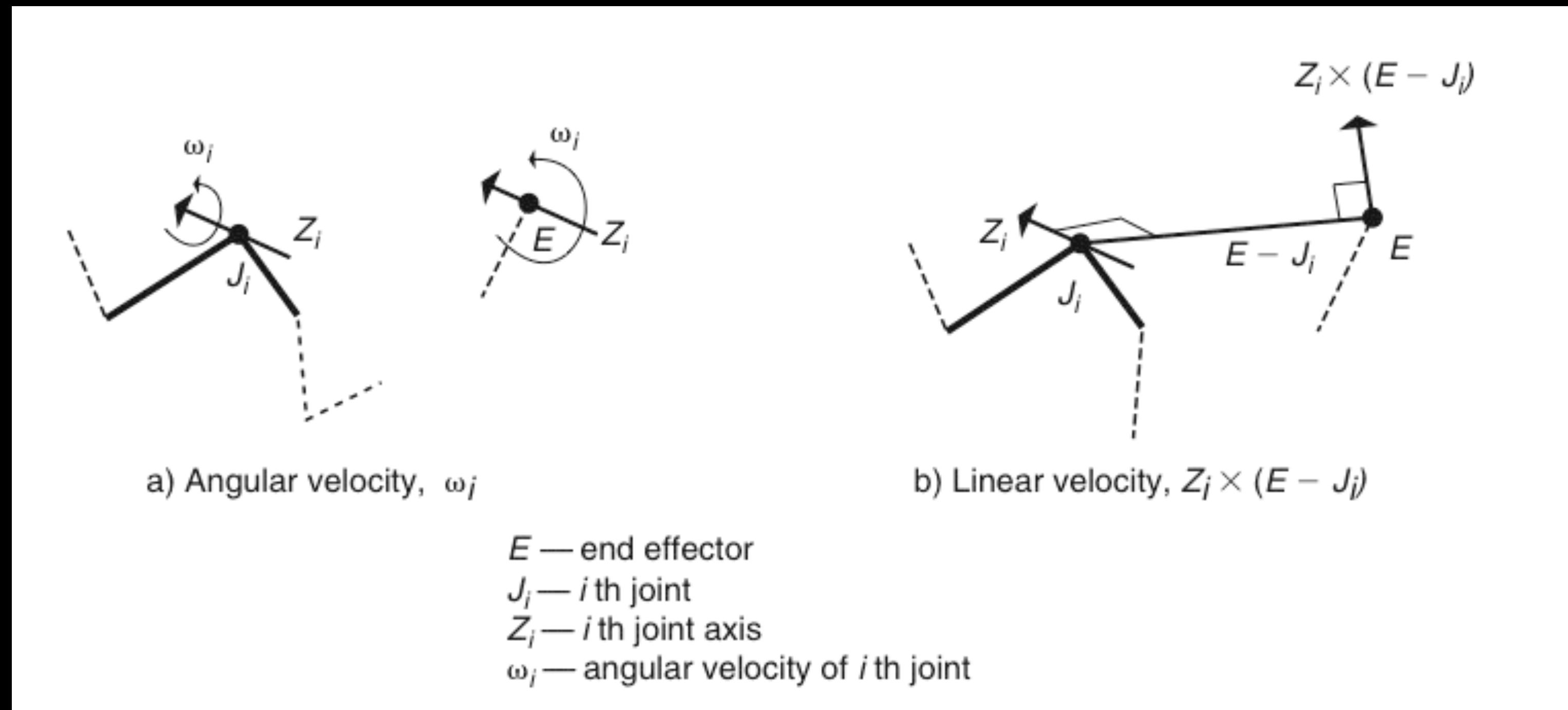
$$V = J(\theta) \dot{\theta}$$

$$V = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z] \quad \dot{\theta} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \dot{\theta}_4, \dot{\theta}_5, \dot{\theta}_6]$$

$$J = \begin{bmatrix} \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} & \frac{\partial p_x}{\partial \theta_6} \\ \frac{\partial p_y}{\partial \theta_1} & & \\ \frac{\partial \alpha_z}{\partial \theta_1} & & \frac{\partial \alpha_z}{\partial \theta_6} \end{bmatrix}$$

Inverse Kinematics - Computing Jacobian

- (Need to convert to world coordinates)

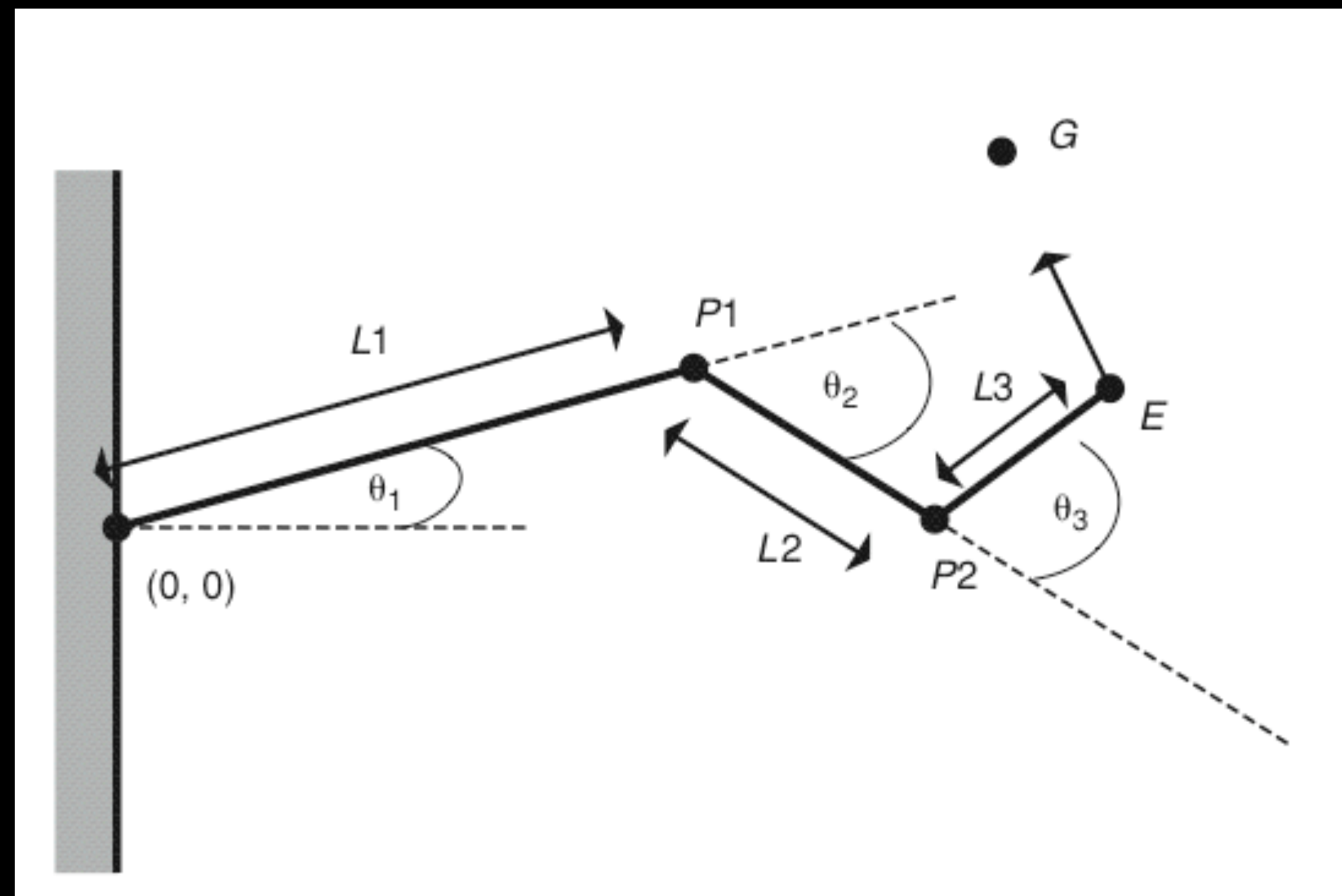


Change in orientation

Change in position

Only valid instantaneously

IK - configuration



Valid only for this moment! - Iteratively Compute!

Summay

- Forward Kinematics
- Inverse Kinematics